

Stochastic Rounding in Low-Precision Transformer Inference: A Variable-Precision Emulation Study of a Small GPT-2

Yohan Chatelain*

*Krembil Centre for Neuroinformatics
Centre for Addiction and Mental Health
Toronto, Canada*

YOHAN.CHATELAIN@CAMH.COM

Pablo de Oliveira Castro*

*Université Paris-Saclay, UVSQ, LI-PaRAD
Versailles, France*

PABLO.OLIVEIRA@UVSQ.FR

Abstract

Should low-precision transformer inference use stochastic rounding (SR) or round-to-nearest (RN)? The answer depends on *where* in the network you look. We isolate this effect by holding the numerical format fixed and varying only the rounding rule at individual operation sites.

To enable experiments at freely chosen precisions, we extend PRISM, a vectorized CPU rounding library, to arbitrary virtual precision via a new variable-precision stochastic rounding (VPSR) algorithm, and prove that the rounding decision is evaluated exactly in hardware floating point.

We develop two analyses that offer complementary insight into the site-level trade-off, without fully modelling end-to-end error propagation. First, a probabilistic forward-error bound for linear projections shows that SR’s error envelope grows as $\mathcal{O}(\sqrt{n}u)$ in the reduction length n , against $\mathcal{O}(nu)$ for RN, a gap that widens rapidly at low precision and is most pronounced in the long multilayer perceptron (MLP) down-projection. Second, a second-order decomposition of the expected cross-entropy loss change at the output softmax into signed drift, drift curvature, and a variance penalty weighted by the Fisher information matrix reveals why the two sites behave oppositely: MLP noise is predominantly a uniform logit shift, to which the softmax is insensitive, so SR’s variance is largely discounted; head noise is genuinely non-uniform across the vocabulary and is not.

These analyses are consistent with the observations. On DistilGPT-2 at $t = 6$ significant bits, SR in the MLP raises the perplexity to $\times 1.15$ the full-precision reference, compared with $\times 2.21$ for RN. At the language-model head, the ordering reverses: SR’s excess loss is driven by its non-uniform variance penalty, whereas deterministic RN carries zero variance penalty. In a mixed-precision configuration (with MLP output at $t = 6$), assigning SR to the MLP and RN to the head brings perplexity within $\times 1.10$ of the full-precision reference, a 28% reduction over matched-bit RN.

Keywords: stochastic rounding, low-precision arithmetic, transformer inference, floating-point emulation, large language models

1 Introduction

Transformer inference increasingly uses formats narrower than the binary64 arithmetic for which much of classical numerical analysis was developed. The dominant response is

*. Equal contribution.

quantization: choose a target format, map weights and activations onto it, and recover accuracy through calibration, outlier handling, or per-channel scaling (Dettmers et al., 2022; Xiao et al., 2023; Lin et al., 2024; Gong et al., 2024). In that program, the rounding operator is usually fixed by the underlying hardware format. In post-training quantization, offline weight-rounding methods such as AdaRound (Nagel et al., 2020) optimize the rounding direction of static weights via a local second-order task loss surrogate. However, the rounding rule used for intermediate calculations during inference has received less attention. In contrast, dynamic rounding is already becoming explicit in low-precision training: NVIDIA’s microscaled NVFP4 recipe, for example, selectively uses stochastic rounding for gradient casts to reduce quantization bias (Abecassis et al., 2025). Whether this same selectivity is useful during inference remains an open question.

This paper isolates that question by holding the numerical format fixed and varying only the rounding rule. Round-to-nearest (Muller et al., 2018) (RN) returns the closer representable neighbor, whereas stochastic rounding (Forsythe, 1959; Croci et al., 2022) (SR), proposed by Forsythe for controlling accumulated roundoff and pioneered in deep learning by Gupta et al. (2015) to prevent gradient stagnation in low-precision training, selects either neighbor with probability proportional to proximity, making each rounding error zero in expectation. RN errors, by contrast, are deterministic functions of their operands and may align; sufficiently small addends can be absorbed by a large accumulator or repeated operations can drift away from the exact result. SR therefore trades systematic local drift for higher variance.

Whether stochastic rounding improves inference accuracy depends first on the operation. A long reduction is more exposed to accumulated RN drift than a short one. In a GPT-2-style block (Radford et al., 2019), the multilayer perceptron (MLP) down-projection reduces over $d_{\text{ff}} = 3072$ terms, while an attention-head score reduces over $d_{\text{head}} = 64$. At the same accumulator precision, these operations can therefore have very different exposure to accumulated roundoff.

The outcome also depends on the operation’s location, both because local input distributions affect how errors initially accumulate, and because downstream layers transform the resulting perturbations. A perturbation in an attention score passes through the softmax before changing the weighted value sum, whereas a perturbation from the MLP down-projection enters the residual stream and is transformed by later blocks. The same change of rounding rule can therefore reduce perplexity at one site and increase it at another. This leads to the research question of this paper: *when and where does stochastic rounding improve transformer inference accuracy over round-to-nearest at low fixed precision?*

We study this question in the GPT-2 architecture, using DistilGPT-2 as a six-block instance, and compare the two rounding rules site by site. Hardware SR is tied to particular formats, operations, and accelerators, so experiments at freely selected precisions require software emulation at every instrumented operation, and the arithmetic intensity of transformer inference requires that emulator to be fast and to vary precision and rounding rule at run time. The contributions of this paper are as follows:

- We present the **variable-precision stochastic rounding (VPSR)** algorithm (Section 3), prove its correctness, and implement it in **PRISM** (Chatelain, 2026), where it

runs up to $70\times$ faster than Verificarlo’s Monte Carlo Arithmetic (MCA) random rounding (Section 4.3).

- We analyze SR in transformer layers (Section 5): we establish a vector Bienaymé–Chebyshev bound that extends prior scalar probabilistic forward-error analysis to the full matrix multiplication of the MLP projection. We also analyze softmax error propagation by deriving an exact decomposition of the cross-entropy loss change into signed target shift (D) and Kullback–Leibler (KL) distortion, and perform a second-order Taylor expansion of the KL term into drift curvature (Q) and variance penalty (V) terms, characterizing the site-level trade-offs between SR and RN.
- We evaluate on DistilGPT-2 and WikiText-2 (Section 6) by lowering t site by site. SR is better in the MLP and RN at the language-model head, and the MLP gap widens as more down-projections are affected. At $t = 6$ in the MLP, SR holds perplexity degradation to 15% over the full-precision reference against 121% for RN, and comes within 0.3% of the reference by $t = 8$.

Disclaimer on AI usage. AI tools (specifically Anthropic Claude, Google Gemini, and OpenAI Codex) were used during the preparation of this manuscript to assist with prose drafting, editing, and checking the mathematical proofs. All derivations, empirical evaluations, and final text were independently verified by the authors, who take full responsibility for the contents of this work.

2 Background and related work

2.1 Stochastic rounding

Let a binary floating-point format have p significand bits, including the implicit leading bit. We write $u = 2^{1-p}$ for its maximum relative ULP scale. Thus $u = 2^{-23}$ for `binary32` ($p = 24$) and $u = 2^{-52}$ for `binary64` ($p = 53$). This u is the spacing at 1 and is twice the conventional RN unit roundoff. For a real x that is not representable on p bits, let $\lfloor x \rfloor$ be the largest representable number below x and $\lceil x \rceil$ the smallest above it, so that $\lfloor x \rfloor < x < \lceil x \rceil$. Write $\text{ulp}(x) = \lceil x \rceil - \lfloor x \rfloor$ for the spacing of the floating-point grid in the binade containing x , which satisfies (Muller et al., 2018)

$$\frac{u}{2} |x| < \text{ulp}(x) \leq u |x|. \quad (1)$$

Round-to-nearest returns whichever of $\lfloor x \rfloor$, $\lceil x \rceil$ is closer, resolving ties by a fixed rule (IEEE, 2019). For non-representable x , stochastic rounding returns:

$$\text{SR}(x) = \begin{cases} \lceil x \rceil, & \text{with probability } p_x = \frac{x - \lfloor x \rfloor}{\lceil x \rceil - \lfloor x \rfloor}, \\ \lfloor x \rfloor, & \text{with probability } 1 - p_x, \end{cases}$$

so that $\mathbb{E}[\text{SR}(x)] = x$. SR commits an error of magnitude at most $\text{ulp}(x)$; RN commits at most $\frac{1}{2}\text{ulp}(x)$. Consequently, for normal results, RN has relative error at most $u/2$ and SR at most u . RN therefore has the smaller worst-case error per operation, while SR is unbiased on average.

Deterministic worst-case analysis of a length- n dot product under RN yields an error bound proportional to nu , and the bound is attained when the individual errors align (Higham, 2002). The mathematical properties of stochastic rounding are well established (Crocì et al., 2022). Under SR the errors form a martingale difference sequence, and probabilistic backward error analysis replaces the nu factor by one of order $\sqrt{nu}$ (Higham and Mary, 2019; Connolly et al., 2021). While this extends to other multilinear kernels (e.g., polynomial evaluation and sum-product trees (de Oliveira Castro et al., 2025)), these bounds do not apply across nonlinear operations. Under RN, an addend small enough relative to the accumulator is absorbed entirely, so a sum of many small equal terms stalls at a value set by u rather than by the terms (Higham, 2002). While stochastic rounding is sometimes criticized for the hardware overhead of pseudorandom number generation, hardware implementations based on linear-feedback shift registers make SR highly efficient in time, energy, and area (Ben Ali et al., 2024).

2.2 Emulating stochastic rounding in software

The standard construction uses *error-free transforms* (EFTs). For an elementary operation $x = a \circ b$, an EFT computes a pair (σ, τ) of representable numbers with $\sigma = \text{fl}(a \circ b)$, where $\text{fl}(\cdot)$ denotes floating-point rounding to nearest, and $\sigma + \tau = x$ exactly, recovering the rounding residual. TwoSum (Dekker, 1971) does this for addition in six operations, and TwoProdFMA (Graillat et al., 2006) does it for multiplication in two when a fused multiply-add is available. Given (σ, τ) , SR reduces to comparing τ against a uniform threshold drawn on $[0, \text{ulp}(\sigma))$.

Verrou (Févotte and Lathuilière, 2016) implemented this scheme, and Fasi and Mikaitis subsequently formalized and analyzed it, giving pseudocode and correctness arguments for all five elementary operations (Fasi and Mikaitis, 2021). A second mechanism is to use extended precision for the intermediate result and then to round back to the working precision. This is implemented in Monte Carlo Arithmetic (Parker, 1997) with the *random rounding* (MCA RR) mode, available as a backend of Verificarlo (Denis et al., 2016). Contrary to the EFT route, MCA RR leaves the virtual precision t free to be chosen by the user, at the cost of evaluating each operation in higher precision.

In deep learning, simulation frameworks such as QPyTorch (Zhang et al., 2019) support stochastic rounding across tensors, but evaluate composite operations (such as matrix multiplications) using an uninstrumented high-precision accumulator (binary32). Capturing error accumulation or stagnation occurring along intermediate reduction chains requires instruction-level emulation instead.

By contrast, compiler tools like Verificarlo (Denis et al., 2016) transparently intercept elementary floating-point operations, allowing PRISM (Chatelain, 2026) to vectorize EFT rounding at the instruction level. However, standard EFT methods round strictly at hardware precision ($t = p$); Section 3 resolves this limitation with the VPSR algorithm, decoupling virtual precision from the hardware format without extended-precision overhead.

While hardware support exists, it is currently restricted to a few architectures and formats/instructions. SR is not an IEEE-754 rounding attribute (IEEE, 2019), and commodity CPUs do not expose it. Graphcore IPU couple random generation to supported floating-point arithmetic (Graphcore, 2020); AWS NeuronCore-v2 exposes an SR/RNE mode for

FP32-to-BF16 autocasting (Amazon Web Services, 2023); and NVIDIA PTX provides `cvt.rs` conversions with caller-supplied random bits on supported Blackwell targets (NVIDIA, 2025). These mechanisms are tied to particular formats, operations, and accelerators. Software emulation is therefore a practical way to study SR on CPUs at a freely selected precision, including precisions that have no hardware format.

2.3 Low-precision transformer inference

The quantization literature attacks a different part of the problem. LLM.int8() (Dettmers et al., 2022) is an 8-bit integer quantization scheme for transformer inference; it isolates the activation outliers that dominate quantization error in large transformers into a higher-precision path. SmoothQuant migrates activation difficulty into the weights through a per-channel rescaling so that both become quantizable (Xiao et al., 2023). AWQ selects salient weight channels by activation statistics and protects them (Lin et al., 2024). An empirical study by Gong et al. (2024) frames all of these as perturbation control and asks which perturbations matter.

All of these hold the rounding rule fixed and design the mapping of values onto the grid. We do the opposite: the mapping is fixed and the rule is the variable. The two address different error sources and can be applied together, and the mechanism studied here, the interaction of rounding choice with network propagation and downstream loss, is invisible to a formulation in which rounding is a black box.

Stochastic rounding in the forward pass. The prevailing consensus confines SR to backward-pass gradient quantization and rejects it for the forward pass. Chmiel et al. (2023) give an explicit mathematical argument: because activation functions are nonlinear, the unbiasedness SR provides at the tensor level ($\mathbb{E}[\text{SR}(x)] = x$) does not survive composition with a nonlinear f , so that $\mathbb{E}[f(\text{SR}(x))] \neq f(x)$. They conclude that SR provides no benefit in the forward pass and that RN, which minimises mean-squared error, should be preferred. NVIDIA’s NVFP4 pretraining recipe (Abecassis et al., 2025) follows the same split, mandating SR only for gradient casts and using RN in the forward path. In the forward pass, standard quantization recipes typically address representation error by managing activation outliers, for example, using random Hadamard transforms (Ashkboos et al., 2024; Abecassis et al., 2025). However, outlier mitigation leaves the arithmetic reduction operator unaddressed: in long inner products, deterministic RN errors still accumulate systematically into directional drift.

Our analysis in Section 5 shows that this categorical rejection of SR in the forward pass is too coarse: the performance of SR versus RN differs across parts of the network. In MLP projections, the long linear reduction allows deterministic RN errors to accumulate into non-uniform drift, whereas SR limits this accumulation and propagates predominantly as a uniform shift that the output softmax ignores (Section 5.2). Conversely, at the output of the network (the language-model head), SR introduces non-uniform fluctuations directly across classes, which cross-entropy loss penalizes, shifting the advantage to RN. A site-selective application of SR—using it at MLP projections while retaining RN at pre-softmax sites—exploits these site-dependent dynamics to control reduction drift directly at the arithmetic level.

2.4 The architecture under study

We use the sequential pre-LayerNorm GPT-2 block (Radford et al., 2019; Vaswani et al., 2017). Each layer first applies its attention residual and then feeds the resulting stream to a separately normalized MLP residual:

$$a_\ell = x_\ell + \text{Attn}_\ell(\text{LN}_{1,\ell}(x_\ell)), \quad x_{\ell+1} = a_\ell + \text{MLP}_\ell(\text{LN}_{2,\ell}(a_\ell)),$$

where $x_\ell, a_\ell \in \mathbb{R}^{T \times d_{\text{model}}}$ are residual streams at layer ℓ for a sequence of T tokens, $\text{LN}_{1,\ell}$ and $\text{LN}_{2,\ell}$ are separate layer normalizations, Attn_ℓ is multi-head self-attention, and MLP_ℓ is the position-wise feedforward network. The normalization sits inside the residual branch rather than after it (Xiong et al., 2020), so the stream x_ℓ is never renormalized and a perturbation introduced at layer ℓ propagates additively to the output.

The MLP is a pair of affine maps around a GELU nonlinearity ϕ (Hendrycks and Gimpel, 2016),

$$z = \tilde{a}W_1 + b_1, \quad h = \phi(z), \quad y = hW_2 + b_2, \quad (2)$$

with $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ and $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$. Attention first forms queries, keys and values by a single projection of the normalized stream, $[Q \mid K \mid V] = \tilde{x}W_{\text{qkv}} + b_{\text{qkv}}$ (the `attn_c_attn` matrix multiplication), splits the result across H heads, and for each head computes

$$S = \frac{QK^\top}{\sqrt{d_{\text{head}}}}, \quad A = \text{softmax}(S), \quad O = AV,$$

where the softmax is taken row-wise under a causal mask. The per-head outputs are concatenated and projected back to the stream by W_o (the `attn_c_proj` matrix multiplication). After the last block, a final linear map $W_{\text{head}} \in \mathbb{R}^{d_{\text{model}} \times V}$ produces logits over a vocabulary of V tokens, which are scored by cross-entropy against the next token.

The dimensions matter for what follows, because they set reduction lengths. The lower panel of Figure 1 places the reductions of one block on the dataflow. Three of the four matrix multiplications reduce over d_{model} (`attn_c_attn`, `attn_c_proj`, `mlp_c_fc`) and the fourth over d_{ff} (`mlp_c_proj`), while inside attention the score product reduces over d_{head} and the value product over the sequence length. The specific dimensions of the evaluated model are detailed in Section 6.

Figure 1 places the block-level dataflow inside the complete DistilGPT-2 evaluation path, from token and position embeddings through the six-block stack to the final normalization, language-model head, cross-entropy, and perplexity.

3 Variable-precision stochastic rounding (VPSR)

This section presents the variable-precision stochastic rounding (VPSR) algorithm, which decouples the rounding decision from the hardware format by decomposing each exact arithmetic result onto a coarser virtual grid. We prove that the rounding decision is evaluated exactly in hardware floating point, with no extended-precision overhead.

At hardware precision, the EFT of Section 2.2 recovers the exact result $x = \sigma + \tau$ with $\sigma = \text{fl}(x)$, and the stochastic rounding algorithm formalized by Fasi and Mikaitis (2021) compares the hardware residual τ against a uniform threshold on $[0, \text{ulp}_{\text{hw}}(\sigma))$.

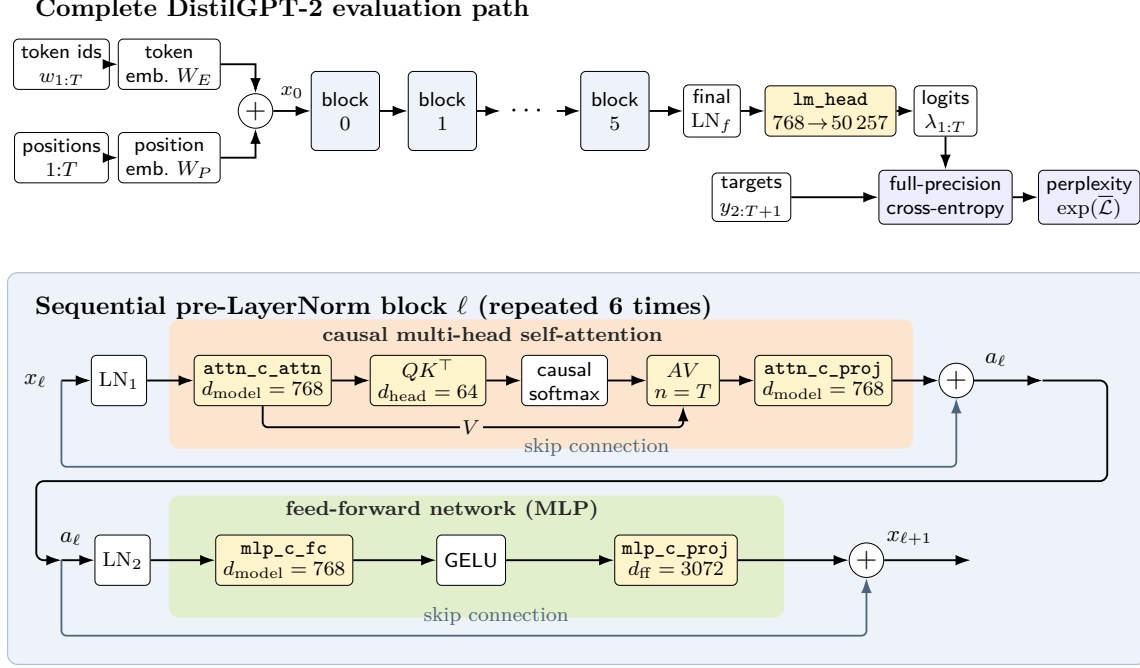


Figure 1: Complete DistilGPT-2 architecture and evaluation path. Token and position embeddings form x_0 , which passes sequentially through six pre-LayerNorm GPT-2 blocks. The lower panel expands the repeated block into its attention and MLP sublayers. Yellow nodes identify the groups instrumented by the rounding experiments.

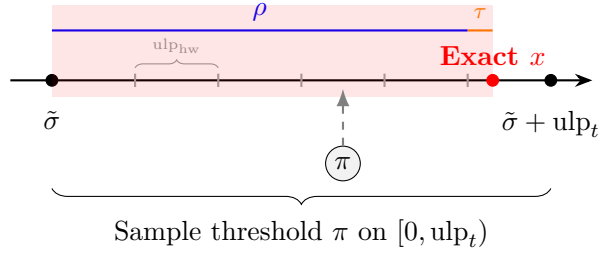


Figure 2: Geometric intuition for variable-precision stochastic rounding (VPSR). An exact value x lies between virtual grid points at precision t . The hardware rounding σ is truncated to $\tilde{\sigma}$, leaving a virtual truncation error ρ and hardware residual τ . A threshold π is sampled on $[0, \text{ulp}_t)$ to determine the virtual rounding direction.

Variable-precision stochastic rounding (VPSR) extends this algorithmic framework to a coarser virtual grid at a precision $t \leq p$ instead, where p is the hardware precision. Truncating the head to that grid, $\tilde{\sigma} = \text{trunc}_t(\sigma)$, splits the distance from x down to the virtual grid point into a virtual truncation error $\rho = \sigma - \tilde{\sigma}$ and the hardware residual τ

Algorithm 1 Variable-precision stochastic rounding

Require: Exact inputs σ, τ , virtual precision t
Ensure: Rounded value r at virtual precision t

```

1:  $\tilde{\sigma} \leftarrow \text{trunc}_t(\sigma)$  ▷ Truncated value at precision  $t$ 
2:  $\rho \leftarrow \sigma - \tilde{\sigma}$  ▷ Exact virtual truncation error
3: if  $\rho = 0$  and  $\tau = 0$  then return  $\tilde{\sigma}$  ▷ Flattened to a mask in SIMD
4:  $s_{\text{dir}} \leftarrow \begin{cases} -1 & \text{if } (\rho = 0 \text{ and } \tau < 0) \text{ or } \rho < 0 \\ 1 & \text{otherwise} \end{cases}$  ▷ Determine rounding direction sign
   ▷ The choice of  $\eta$  ensures  $\text{ulp}_t$  will be the ULP of the exact value  $x$  at precision  $t$ , i.e.,  $\text{ulp}_t(x)$ 
5:  $\eta \leftarrow \begin{cases} \text{exponent}(\text{predecessor\_abs}(\tilde{\sigma})) & \text{if } (s_{\text{dir}} < 0) \neq (\tilde{\sigma} < 0) \\ \text{exponent}(\tilde{\sigma}) & \text{otherwise} \end{cases}$ 
6:  $\text{ulp}_t \leftarrow s_{\text{dir}} \cdot 2^{\eta - (t-1)}$  ▷ Compute signed ULP at virtual precision  $t$  using exponent  $\eta$ 
7:  $S \leftarrow \begin{cases} 2^{64} & \text{if } \eta - (t-1) \leq e_{\min} \\ 1 & \text{otherwise} \end{cases}$  ▷ Scale subnormals to normal range
8:  $\rho_S \leftarrow \rho \cdot S, \quad \tau_S \leftarrow \tau \cdot S, \quad \text{ulp}_{t,S} \leftarrow \text{ulp}_t \cdot S$ 
9: Sample  $z$  uniformly from  $\{k2^{-r} : 0 \leq k < 2^r\}$ 
10:  $\pi \leftarrow \text{ulp}_{t,S} \cdot z$  ▷ Random threshold
11:  $D \leftarrow (\rho_S - \pi) + \tau_S$  ▷ Decision boundary term
12:  $\Delta \leftarrow \begin{cases} \text{ulp}_t & \text{if } D \cdot s_{\text{dir}} \geq 0 \\ 0 & \text{otherwise} \end{cases}$ 
13: return  $\tilde{\sigma} + \Delta$ 
    
```

(Figure 2),

$$x = \tilde{\sigma} + \rho + \tau.$$

To perform ideal stochastic rounding, [Algorithm 1](#) draws a continuous threshold $\pi \sim \mathcal{U}[0, \text{ulp}_t)$ and rounds away from $\tilde{\sigma}$ when the signed threshold does not exceed the signed distance $\rho + \tau$. Fixing $\pi = \frac{1}{2}\text{ulp}_t$ instead gives untied round-to-nearest at precision t .

Because hardware generators provide discrete bits rather than continuous draws, PRISM approximates the uniform distribution using r random bits ($r = 24$ for **binary32** and $r = 53$ for **binary64**). Let $\text{SR}_{t,r}$ denote this finite-bit implementation. This discretization introduces a negligible bias bounded by the number of random bits r :

$$|\mathbb{E}[\text{SR}_{t,r}(x)] - x| \leq 2^{-r} |\text{ulp}_t(x)|.$$

Three steps that are trivial at hardware precision require care at virtual precision:

1. **Two-term decision.** The relevant distance from x to the virtual grid point is $\rho + \tau$, not τ alone. These terms can differ by the full hardware precision, making the sign of their floating-point sum fragile. [Section 3.1](#) proves that evaluating $(\rho - \pi) + \tau$ in hardware arithmetic nevertheless preserves the exact sign, requiring no compensated arithmetic.
2. **Power-of-two boundaries.** When the rounding direction carries x across a binade boundary, line 5 of [Algorithm 1](#) selects the predecessor exponent so that ulp_t reflects the grid spacing at x rather than at $\tilde{\sigma}$.

3. **Subnormal virtual grid.** When ulp_t is subnormal, the threshold product $\text{ulp}_t \cdot z$ loses precision. Lines 7–8 scale all quantities by $S = 2^{64}$ (exact as a power of two) and compare in the normal range; the decision is invariant under common positive scaling.

3.1 Exactness of the rounding decision

We assume IEEE 754 floating-point arithmetic with round-to-nearest. Let $\text{fl}(x + y)$ and $\text{fl}(x - y)$ denote floating-point addition and subtraction, respectively.

We define the rounding direction sign s_{dir} (equivalently to line 4 of [Algorithm 1](#), noting that the $\rho = 0, \tau = 0$ case is handled on line 3):

$$s_{\text{dir}} = \begin{cases} 1 & \text{if } \rho > 0 \text{ or } (\rho = 0 \text{ and } \tau > 0) \\ -1 & \text{if } \rho < 0 \text{ or } (\rho = 0 \text{ and } \tau < 0) \end{cases}$$

Throughout the theorem and its proof, ulp_{hw} abbreviates $\text{ulp}_{\text{hw}}(x)$, the hardware grid spacing at x . This is an absolute spacing, not the relative ULP scale $u = 2^{1-t}$ of [Eq. \(1\)](#).

Theorem 1 (Exact sign evaluation of the rounding decision) *Let ρ , π , and τ be floating-point numbers. If the following properties hold:*

1. $|\tau| \leq \text{ulp}_{\text{hw}}/2$
2. $\rho = 0$ or $|\rho| \geq \text{ulp}_{\text{hw}}$
3. ρ and π share the same sign

Then the floating-point evaluation of the decision boundary preserves the sign:

$$(\rho - \pi + \tau) \cdot s_{\text{dir}} \geq 0 \iff \text{fl}(\text{fl}(\rho - \pi) + \tau) \cdot s_{\text{dir}} \geq 0$$

The three hypotheses hold in [Algorithm 1](#): (1) is the standard RN bound $|\tau| \leq \text{ulp}_{\text{hw}}/2$; (2) holds because truncation zeros trailing bits without changing the exponent, making ρ an integer multiple of $\text{ulp}_{\text{hw}}(\sigma)$; since IEEE 754 monotonicity gives $|x| \geq 2^k \Rightarrow |\text{fl}(x)| \geq 2^k$, the rounded value σ cannot cross a binade boundary toward zero, so $\text{ulp}_{\text{hw}}(\sigma) \geq \text{ulp}_{\text{hw}}$; (3) follows from $|\rho| \geq \text{ulp}_{\text{hw}} \geq 2|\tau|$, which prevents τ from flipping the sign of ρ , combined with the algorithm assigning π the sign of s_{dir} . The subnormal scaling on lines 7–8 preserves all three by multiplying distances and spacing by a common exact power of two.

Proof sketch We normalize the variables by the rounding direction sign $s_{\text{dir}} \in \{-1, 1\}$, setting $\rho' = \rho \cdot s_{\text{dir}}$, $\pi' = \pi \cdot s_{\text{dir}}$, and $\tau' = \tau \cdot s_{\text{dir}}$, so that the equivalence becomes $(\rho' - \pi') + \tau' \geq 0 \iff \text{fl}(\text{fl}(\rho' - \pi') + \tau') \geq 0$.

When $\rho = 0$, the first subtraction $\text{fl}(0 - \pi') = -\pi'$ is exact in IEEE 754 arithmetic. The decision then reduces to a single floating-point addition $\text{fl}(-\pi' + \tau')$, whose sign matches the exact sign by monotonicity.

When $\rho \neq 0$, monotonicity directly ensures the forward implication. For the backward implication, the only scenario where the signs could disagree is an artificial collapse to zero, $\text{fl}(\text{fl}(\rho' - \pi') + \tau') = 0$, while the exact sum is strictly negative, $(\rho' - \pi') + \tau' < 0$. Under the bounds from Hypotheses 1 and 2, such a collapse requires the intermediate difference $\text{fl}(\rho' - \pi')$ to equal $-\tau'$, forcing $\rho'/2 \leq \pi' \leq 2\rho'$. This condition satisfies the hypotheses of

Sterbenz’s Lemma (Sterbenz, 1974), which guarantees that the floating-point subtraction $\text{fl}(\rho' - \pi')$ is exact. Exact subtraction then yields $(\rho' - \pi') + \tau' = 0$, contradicting the assumption that the exact sum is strictly negative. Thus, no sign discrepancy can occur. The complete formal proof is deferred to Section A. ■

4 PRISM: vectorized probabilistic rounding at variable precision

This section describes how Algorithm 1 is applied to every floating-point operation of a PyTorch model and what it costs. We implement VPSR in PRISM (*Probabilistic Rounding with Instruction Set Management*), an open-source C++17 library that previously applied vectorized SR only at hardware precision (González-Pepe et al., 2026). The Verificarlo compiler pass (Denis et al., 2016) replaces each floating-point instruction of the compiled framework with a call to a PRISM kernel, which computes the exact result as an EFT pair and rounds it at a virtual precision t taken from a run-time configuration (Figure 3). In compiler-instrumented loops, PRISM SR costs $8.5\text{--}32.1\times$ native time, $4.4\text{--}70.5\times$ less than Verificarlo’s MCA random rounding (Section 4.3).

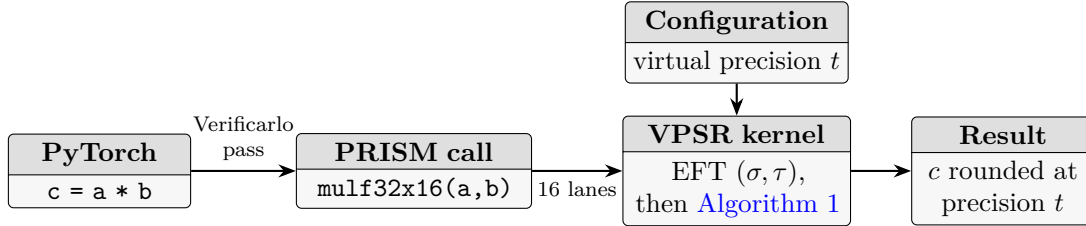


Figure 3: Execution of one instrumented operation. At compile time, the Verificarlo pass replaces a 512-bit `binary32` multiplication with a call to the PRISM entry point for 16 lanes. At run time, the kernel rounds all lanes with Algorithm 1 at the precision set.

4.1 Instrumentation and kernels

The Verificarlo pass maps each scalar or vector floating-point instruction to a PRISM entry point for the same operation, format and lane count, so PyTorch’s source is unchanged. Each call rounds all its lanes in one kernel invocation. We use PRISM’s static build, which fixes the instruction set at compile time.

Each kernel computes the EFT pair (σ, τ) of Section 2.2 and then applies Algorithm 1. SR and RN share the kernel: SR draws z from r random bits, and RN fixes $z = 1/2$. Because the final comparison of Algorithm 1 is non-strict, an exact midpoint rounds away from $\tilde{\sigma}$, and hence away from zero; giving *untied* RN. FMA is rounded as one operation, using the `ErrFmaNearest` transform (Boldo and Muller, 2011) to represent $ab + c$ exactly.

The kernels are branch-free. Every data-dependent condition, in Algorithm 1 and in the EFTs, is computed as a lane mask and resolved by selection, so all lanes follow one instruction

Table 1: Slowdown of PRISM SR versus MCA RR relative to native execution ($N = 65,536$, 30 runs; $t = 24$ for `binary32`, $t = 53$ for `binary64`). PRISM and MCA entries are the median ratios to native time for the same operation and data type (lower is better); MCA/PRISM is the median per-run ratio of MCA to PRISM time.

Operation	binary32			binary64		
	PRISM	MCA	MCA/PRISM	PRISM	MCA	MCA/PRISM
add	31.1	142.4	4.7	11.6	115.6	10.0
mul	32.1	143.7	4.5	11.3	163.1	14.6
div	25.6	112.7	4.4	12.0	229.9	19.2
fma	19.1	88.3	4.7	8.5	597.4	70.5

stream whatever their values, including infinities and NaNs. The kernels are written once against Google Highway (Google, 2024), a C++ SIMD abstraction, and compile to its x86, Arm, RISC-V and WebAssembly targets; where no hardware FMA exists, `TwoProdFMA` uses the emulated FMA of Graillat and Muller (2023).

4.2 Run-time configuration

The virtual precisions for `binary32` and `binary64`, the rounding mode and the generator state are run-time settings, so changing them requires no recompilation. In our experiments, PyTorch module hooks change them before and after each target module (Section 6.1). Kernels read the settings from a per-thread cache. A process-wide setter publishes new settings and advances a configuration epoch, and each worker thread reloads its cache when it observes a new epoch on its next operation, so a change made from the Python thread reaches all of PyTorch’s worker threads.

4.3 Performance

Cost model. Emulation replaces one instruction with an EFT (two to six operations), about twenty-five vector operations for the rounding decision, and part of a random draw. The control flow is fixed, so this cost does not depend on the operands. The slowdown relative to native execution is therefore smaller when native execution is memory-bound and larger when a call covers few lanes.

Benchmark. We time elementwise addition, multiplication, division and FMA over arrays of 65,536 values in `binary32` and `binary64`. Each loop is compiled with `-O3` for 512-bit AVX-512 vectors in three builds: without instrumentation (the native baseline), with the PRISM SR pass, and with Verificarlo’s MCA QUAD backend in random-rounding (RR) mode (Section 2.2). We repeat the measurement 30 times on Intel Xeon Gold 6130 @ 2.1 GHz CPU (cache sizes: 32 KB L1, 1 MB L2, 22 MB L3 shared).

Results. PRISM SR incurs an overhead of $\times 8.5$ – 32.1 , whereas MCA RR incurs $\times 88$ – 597 (Table 1). Compared to MCA RR, PRISM SR is up to $\times 4.7$ faster than for `binary32` operations and up to $\times 70.5$ faster for `binary64`. The gap comes from how each backend

obtains the exact result it rounds. MCA RR computes the result in a wider format and adds its random perturbation there: in `binary64` for `binary32` operations, which the hardware supports, and in `binary128` for `binary64` operations, which most CPUs emulate in software. PRISM recovers the exact result with EFTs in the working format, so both formats use hardware instructions only. Performance is nearly independent of the virtual precision: across precisions, PRISM times vary by less than 1% and MCA times by at most 5%.

5 Theoretical error analysis of SR versus RN

The GPT-2 architecture contains projections with very different reduction lengths—the MLP down-projection accumulates over $n = 3072$ terms, whereas the attention projections accumulate over $n = 768$ —and the language-model head applies the softmax directly to the perturbed logits. These structural differences create distinct opportunities for SR’s variance–drift trade-off. This section analyzes both regimes: first, establishing a probabilistic forward error bound for the affine projection layers (`attn_c_proj` and `mlp_c_proj`) via a vector Bienaymé–Chebyshev construction (Section 5.1); second, deriving an analytical decomposition of cross-entropy loss change into drift and variance components at the output softmax (Section 5.2); and third, empirically validating this trade-off on DistilGPT-2 (Section 5.3). Throughout, statements concern the precision at which a reduction *accumulates*. On physical hardware, a kernel nominally in `bfloat16` may store in `bfloat16`, multiply into a wider product, and accumulate in `binary32`; under PRISM, every elementary operation rounds at the same virtual precision, $u_{\text{sto}} = u_{\text{mul}} = u_{\text{acc}} = 2^{1-t}$, so the relative ULP scale u below is 2^{1-t} for the significand precision t swept by our harness.

5.1 Forward error analysis of linear projection layers

The multilayer perceptron (MLP) block in the GPT-2 architecture consists of two consecutive linear projections surrounding an element-wise activation function. Following the row-vector convention of Eq. (2),

$$\text{MLP}(x) = \phi(xW_1 + b_1)W_2 + b_2,$$

where ϕ is the GELU activation function (Hendrycks and Gimpel, 2016), $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ is the expansion matrix (`mlp_c_fc`), and $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ is the projection matrix (`mlp_c_proj`). In DistilGPT-2, $d_{\text{model}} = 768$ and $d_{\text{ff}} = 3072$.

Limitations of backward error frameworks for nonlinear activations. The backward error framework of Beuzeville et al. (2026) analyzes multi-layer networks by absorbing rounding errors into equivalent weight perturbations ΔW . For a layer followed by an activation ϕ , the componentwise backward error bound includes a term proportional to $l_\phi/\kappa_\phi(z)$, where $\kappa_\phi(z) = |z\phi'(z)/\phi(z)|$ is the relative condition number of ϕ and $l_\phi u$ bounds the evaluation error of ϕ .

For GELU, this framework does not yield a uniform backward guarantee due to the vanishing derivative at its local minimum. Specifically, at $z^* \approx -0.752$, $\phi'(z^*) = 0$ while $\phi(z^*) \approx -0.170 \neq 0$. Consequently, the condition number vanishes ($\kappa_\phi(z^*) = 0$), causing the backward error bound $l_\phi/\kappa_\phi(z)$ to diverge in the neighborhood of z^* .

Although each ideal SR step is conditionally unbiased, propagating SR perturbations through a nonlinear activation generally introduces composition bias (El Arar et al., 2024,

Section 4). To isolate the reduction error from this nonlinear composition, we focus on the affine down-projection `mlp_c_proj` (and, by extension, the attention projection `attn_c_proj`):

$$y = hW + b,$$

where $h \in \mathbb{R}^n$ is the input row vector, $W \in \mathbb{R}^{n \times m}$ is the projection matrix, $b \in \mathbb{R}^m$ is the bias vector, and $y \in \mathbb{R}^m$ is the output ($m = d_{\text{model}} = 768$). For `mlp_c_proj`, $n = d_{\text{ff}} = 3072$ and $h = \phi(xW_1 + b_1)$; for `attn_c_proj`, $n = d_{\text{model}} = 768$ and h is the concatenated multi-head attention output. Because these projection layers have no subsequent activation ($\phi = \text{id}$), the forward error $e = \hat{y} - y$ directly quantifies the perturbation injected into the residual stream.

Error model. Each component of y is evaluated as a left-to-right sequential inner product followed by a bias addition:

$$y_i = \sum_{j=1}^n h_j W_{ji} + b_i, \quad i = 1, \dots, m,$$

totaling $2n$ operations per output component with a longest error-product chain of $n + 1$ rounding factors (one multiplication, $n - 1$ accumulations, and the final bias addition).

Let $u = 2^{1-t}$ denote the relative ULP scale at virtual significand precision t . Assuming no overflow, underflow, or NaNs occur, every elementary operation satisfies $\text{fl}(a \text{ op } b) = (a \text{ op } b)(1 + \delta)$, where:

- under RN, $|\delta| \leq u/2$;
- under SR (mode-1, nearness), $|\delta| \leq u$, with the mean-independence property $\mathbb{E}[\delta_k \mid \delta_1, \dots, \delta_{k-1}] = 0$ (Connolly et al., 2021, Lemma 5.2).

Deterministic forward error bound (RN). Under deterministic RN, rounding errors may align in the worst case. Applying Higham’s exact forward error bound for inner products (Higham, 2002, Lemma 3.1), each component and then the vector satisfy

$$\begin{aligned} |\hat{y}_i - y_i| &\leq \left((1 + u/2)^{n+1} - 1 \right) \left(\sum_{j=1}^n |h_j| |W_{ji}| + |b_i| \right), \\ \|\hat{y} - y\|_2 &\leq \left((1 + u/2)^{n+1} - 1 \right) (\|h\|_1 \|W\|_1 + \|b\|_1). \end{aligned}$$

In the high-precision regime where $nu \ll 1$, the coefficient linearizes to $\approx \frac{n+1}{2}u$; when $nu \gg 1$, it grows exponentially, reaching $\approx 8.11 \times 10^{80}$ at $t = 4$ for $n = 3072$.

Probabilistic rounding and scalar bounds. In the first-order regime, mean-independence allows SR forward error bounds proportional to $\mathcal{O}(\sqrt{n}u)$ rather than the $\mathcal{O}(nu)$ deterministic RN bound. Existing probabilistic frameworks for bounding scalar inner product errors $e_i = \hat{y}_i - y_i$ follow one of two strategies:

- *Azuma–Hoeffding (AH) method* (Connolly et al., 2021; Ipsen and Zhou, 2020; de Oliveira Castro et al., 2025): Martingale bounds provide exponential probability tails, which tolerate a union bound over dimensions with only a mild $\mathcal{O}(\sqrt{\ln m})$ penalty. However, the bound on the increments contains a factor $(1 + u)^{n+1}$ (El Arar et al., 2024, Section 7). When $nu \gg 1$, this factor explodes exponentially, rendering AH uninformative at low precisions.

- *Bienaymé–Chebyshev (BC) method* (El Arar et al., 2023, 2024): Bounding the variance of the accumulated error avoids this exponential explosion: because the precision enters squared in the variance bound, $(1 + u^2)^{n+1} - 1 \approx (n + 1)u^2$ remains small even at low precisions where $nu \gg 1$.

The core variance bound for error products under SR is given by the following lemma.

Lemma 2 (El Arar et al., 2023, Lemma 3.1) *Let $\varphi = \prod_{k=1}^K (1 + \delta_k)$, where δ_k are SR-nearness rounding errors satisfying $|\delta_k| \leq u$ and $\mathbb{E}[\delta_k \mid \delta_1, \dots, \delta_{k-1}] = 0$. Then $\mathbb{E}[\varphi] = 1$ and $\text{Var}(\varphi) \leq (1 + u^2)^K - 1$.*

Applying Chebyshev’s inequality to each scalar component e_i yields an error bound scaling with $\frac{1}{\sqrt{\alpha}} \sqrt{(1 + u^2)^{n+1} - 1}$.

Vector L_2 -norm bound: combining BC with a multivariate Markov bound. Extending the scalar analysis to the full output vector $e \in \mathbb{R}^m$ exposes a limitation in existing bounds: AH survives the union bound across m components but fails at low precision t , whereas scalar BC remains stable at low precision but grows rapidly under the union bound. Because Chebyshev’s inequality has a polynomial tail, setting the per-component failure probability to α/m inflates the confidence multiplier by $\sqrt{m}$, from $1/\sqrt{\alpha} \approx 4.47$ to $\sqrt{m}/\sqrt{\alpha} \approx 124$ ($m = 768$, $\alpha = 0.05$). This penalty inflates the bound to $\approx 6870u$, wiping out SR’s advantage over RN.

To circumvent this limitation, our approach combines the low-precision robustness of BC with a single multivariate Markov step applied directly to the expected squared L_2 -norm $\mathbb{E}[\|e\|_2^2]$. This bypasses the union bound, keeping the confidence multiplier $1/\sqrt{\alpha}$ independent of the output dimension m .

Theorem 3 (Vector Bienaymé–Chebyshev Bound) *Under SR-nearness, the forward error $e = \hat{y} - y$ of the projection $y = hW + b$ satisfies, for any $\alpha \in (0, 1)$, with probability at least $1 - \alpha$:*

$$\|\hat{y} - y\|_2 \leq \frac{1}{\sqrt{\alpha}} \sqrt{(1 + u^2)^{n+1} - 1} (\|h\|_2 \|W\|_2 + \|b\|_2).$$

The confidence multiplier $1/\sqrt{\alpha}$ is independent of the output dimension m .

Proof The error in output component i is

$$e_i = \sum_{j=1}^n h_j W_{ji} (\varphi_{ij} - 1) + b_i (\varphi_{b,i} - 1),$$

where φ_{ij} and $\varphi_{b,i}$ represent the error products along the accumulation chains, containing at most $n + 1$ and 1 rounding factors, respectively. By mean-independence, $\mathbb{E}[\varphi_{ij}] = 1$, $\mathbb{E}[\varphi_{b,i}] = 1$, and $\mathbb{E}[e_i] = 0$.

Although the terms $\varphi_{ij} - 1$ share common rounding errors across additions, Minkowski’s inequality in L_2 bounds the standard deviation of their linear combination:

$$\sqrt{\text{Var}(e_i)} \leq \sum_{j=1}^n |h_j| |W_{ji}| \sqrt{\text{Var}(\varphi_{ij})} + |b_i| \sqrt{\text{Var}(\varphi_{b,i})}.$$

Applying [Lemma 2](#) with $\text{Var}(\varphi_{ij}) \leq (1 + u^2)^{n+1} - 1$ and $\text{Var}(\varphi_{b,i}) \leq u^2 \leq (1 + u^2)^{n+1} - 1$ yields

$$\text{Var}(e_i) \leq \left((1 + u^2)^{n+1} - 1 \right) \left(\sum_{j=1}^n |h_j| |W_{ji}| + |b_i| \right)^2.$$

By linearity of expectation, the expected squared L_2 -norm satisfies

$$\mathbb{E}[\|e\|_2^2] = \sum_{i=1}^m \text{Var}(e_i) \leq \left((1 + u^2)^{n+1} - 1 \right) (\|h\| |W| + \|b\|)_2^2.$$

Applying Markov’s inequality to the squared L_2 -norm $\|e\|_2^2$ (which constitutes the multi-variate Bienaymé–Chebyshev inequality) gives $\mathbb{P} \left(\|e\|_2^2 \geq \frac{1}{\alpha} \mathbb{E}[\|e\|_2^2] \right) \leq \alpha$. Taking the square root completes the proof. $\blacksquare$

Numerical evaluation on DistilGPT-2. While [Theorem 3](#) holds for any affine projection of reduction length n , we evaluate the bounds concretely on the feedforward down-projection `mlp_c_proj` ($n = d_{\text{ff}} = 3072$, $m = d_{\text{model}} = 768$) at 95% confidence ($\alpha = 0.05$, $1/\sqrt{\alpha} = \sqrt{20} \approx 4.47$). Because `mlp_c_proj` has four times the reduction length of `attn_c_proj` ($n = 768$), it has the longest accumulation chains and therefore tests the error bounds in the most demanding regime. In the limiting first-order regime $u \rightarrow 0$ (equivalently, $t \rightarrow \infty$):

- RN deterministic: $(1 + u/2)^{n+1} - 1 \approx \frac{n+1}{2}u \approx 1537u$;
- SR vector BC: $\frac{1}{\sqrt{\alpha}} \sqrt{(1 + u^2)^{n+1} - 1} \approx \sqrt{20} \sqrt{n+1} u \approx 248u$.

The SR coefficient is asymptotically smaller than the RN coefficient by a factor of approximately 6.2. At finite precision the exact ratio approaches this limit from above: it is 14.37 at $t = 11$ and 6.35 at $t = 16$.

As shown in [Figure 4](#), the deterministic RN bound suffers from exponential error compounding at low precision ($nu \gg 1$), reaching 2.43×10^{10} at $t = 7$ and 8.11×10^{80} at $t = 4$. In contrast, because precision enters squared under stochastic rounding, the SR vector BC bound grows much more slowly (4.73 at $t = 7$). At $t = 11$ the exact RN and SR coefficients are 3.482 and 0.242, a ratio of 14.37; the ratio decreases toward 6.2 only in the limit $t \rightarrow \infty$.

For fixed α in the first-order regime, [Theorem 3](#) yields an $\mathcal{O}(\sqrt{nu})$ bound for SR, whereas deterministic RN scales as $\mathcal{O}(nu)$. Both bounds are worst-case over the inputs and normalize by $\|h\| |W| + \|b\|_2$, but they treat arithmetic errors differently. Under RN, rounding errors are deterministic functions of the data that can align constructively in the worst case. Under SR, mean-independence yields the $\sqrt{n}$ factor via the Bienaymé–Chebyshev variance bound. While both results are upper bounds, and realized RN errors rarely achieve worst-case alignment, the comparison illustrates the fundamental contrast between the two modes: deterministic rounding permits systematic linear drift along the reduction chain, whereas stochastic rounding restricts error growth to a sub-linear $\mathcal{O}(\sqrt{nu})$ envelope.

The downstream captures in [Section 5.3](#) show larger RN than SR sample logit drift after reducing `mlp_c_proj`. These propagated measurements do not directly test the single-reduction bounds above.

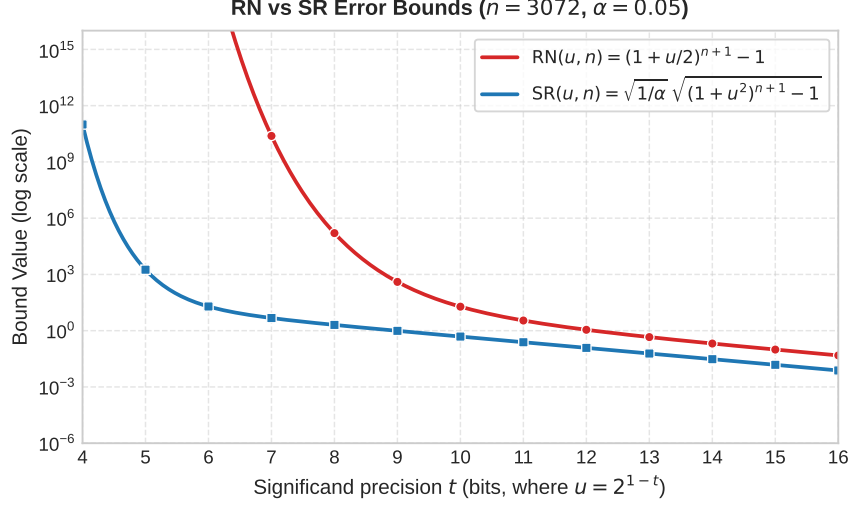


Figure 4: Deterministic RN and SR forward-error bound coefficients for `mlp_c_proj` in DistilGPT-2 ($n = 3072$).

5.2 Error propagation and loss decomposition at the output softmax

Section 5.1 bounds the perturbation that low-precision arithmetic injects into one layer. At the network output, the error travels to the logits and alters each token’s cross-entropy loss. This subsection analyzes that interface to identify, term by term, why SR and RN affect the language-model head differently.

Exact decomposition of the cross-entropy loss change. Fix one token with reference logits $\lambda \in \mathbb{R}^V$, target class y , and predictive distribution $p = \text{softmax}(\lambda)$. Its cross-entropy loss is

$$\mathcal{L}(\lambda) = -\lambda_y + \log \sum_{j=1}^V e^{\lambda_j}. \quad (3)$$

Let the low-precision run perturb logit j by Δ_j . Substituting $\lambda + \Delta$ into Eq. (3) and subtracting gives the exact signed loss change

$$\delta\mathcal{L}(\Delta) := \mathcal{L}(\lambda + \Delta) - \mathcal{L}(\lambda) = -\Delta_y + \log \sum_{j=1}^V p_j e^{\Delta_j}.$$

Let $\tilde{p} = \text{softmax}(\lambda + \Delta)$ and let $D_{\text{KL}}(p \parallel \tilde{p})$ denote the Kullback–Leibler (KL) divergence from p to $\tilde{p}$. Direct substitution gives

$$D_{\text{KL}}(p \parallel \tilde{p}) = \sum_{j=1}^V p_j \log \frac{p_j}{\tilde{p}_j} = \log \sum_{j=1}^V p_j e^{\Delta_j} - p^\top \Delta,$$

and therefore, with $g := p - e_y$, the exact decomposition

$$\delta\mathcal{L}(\Delta) = g^\top \Delta + D_{\text{KL}}(p \parallel \tilde{p}). \quad (4)$$

The linear term is signed and records the target-specific shift, whereas the KL term is non-negative and records the change in the full predictive distribution.

Local quadratic form of the KL term. The KL divergence satisfies $D_{\text{KL}}(p \| p) = 0$ and has vanishing gradient at $\Delta = 0$. Its local quadratic approximation is governed by the Fisher information matrix of the categorical distribution parameterized by logits,

$$H = \text{diag}(p) - pp^\top,$$

which coincides with the Hessian of the cross-entropy loss $\nabla_\lambda^2 \mathcal{L}(\lambda)$ (Martens, 2020, Section 6). Although the KL divergence is asymmetric, both orientations share the same second-order expansion at $\Delta = 0$:

$$D_{\text{KL}}(p \| \tilde{p}) = \frac{1}{2} \Delta^\top H \Delta + r, \quad r = \mathcal{O}(\|\Delta\|^3). \quad (5)$$

Because H is the covariance matrix of a single categorical draw with probabilities p , for any perturbation $\Delta \in \mathbb{R}^V$ the quadratic form equals the variance across classes:

$$\Delta^\top H \Delta = \sum_{j=1}^V p_j \Delta_j^2 - (p^\top \Delta)^2 = \text{Var}_p(\Delta) \geq 0, \quad (6)$$

confirming that $H \succeq 0$ and that the local KL penalty charges variations in Δ across classes considered plausible by p . Fisher information similarly guides parameter sensitivity in post-training quantization (e.g., SqueezeLLM, Kim et al., 2024); in logit space, H arises analytically to penalize non-uniform operational perturbations directly.

An exact consequence of the logit formulation is *shift invariance* (Blanchard et al., 2021): any uniform perturbation $\Delta = c\mathbf{1}$ leaves the predictive distribution identical ($\tilde{p} = \text{softmax}(\lambda + c\mathbf{1}) = p$), so that $D_{\text{KL}}(p \| \tilde{p}) = 0$. Likewise, $g^\top(c\mathbf{1}) = c(p - e_y)^\top \mathbf{1} = 0$, ensuring that $\delta \mathcal{L}(c\mathbf{1}) = 0$ exactly. The one-dimensional subspace $\text{span}(\mathbf{1})$ constitutes the null space of the Fisher metric ($H\mathbf{1} = \mathbf{0}$).

The two probability spaces. Conditional on a fixed token w , two distinct probability spaces arise for SR and RN.

- (i) Under SR, every rounded elementary operation along the trajectory draws fresh pseudo-random bits. This operational randomness makes the logit perturbation $\Delta_{\text{SR}}^{(w)}$ a random vector, driven by elementary rounding errors that satisfy the mean-independence of Section 5.1.
- (ii) Under RN, every operation returns one deterministic value: there is no operational randomness. Conditional on a fixed token,

$$\mathbb{E}_{\text{RN}}[\Delta_{\text{RN}}^{(w)} | w] = \Delta_{\text{RN}}^{(w)}, \quad \Sigma_{\text{RN}}^{(w)} := \text{Cov}_{\text{RN}}(\Delta_{\text{RN}}^{(w)} | w) = 0. \quad (7)$$

Drift–variance decomposition. For either mode $m \in \{\text{SR}, \text{RN}\}$, decompose the logit perturbation as

$$\Delta_m = b_m + \xi_m, \quad \mathbb{E}_m[\xi_m] = 0, \quad (8)$$

where $b_m := \mathbb{E}_m[\Delta_m]$ is the drift and ξ_m is the centered fluctuation, with covariance $\Sigma_m := \text{Cov}_m(\xi_m)$. By [Eq. \(7\)](#), on a fixed token RN carries $b_{\text{RN}} = \Delta_{\text{RN}}$ and $\Sigma_{\text{RN}} = 0$: its entire perturbation is drift.

Substituting $\Delta_m = b_m + \xi_m$ into the local KL expansion [Eq. \(5\)](#) and taking expectations $\mathbb{E}_m[\cdot]$ over operational randomness (over rounding draws for SR, and degenerate for deterministic RN), the linear term in [Eq. \(4\)](#) yields $\mathbb{E}_m[g^\top \Delta_m] = g^\top b_m$. The quadratic term expands as

$$\Delta_m^\top H \Delta_m = b_m^\top H b_m + 2b_m^\top H \xi_m + \xi_m^\top H \xi_m.$$

Taking expectations, the cross term vanishes since $\mathbb{E}_m[\xi_m] = 0$, and the quadratic fluctuation term evaluates to $\mathbb{E}_m[\xi_m^\top H \xi_m] = \text{tr}(H \Sigma_m)$. Thus the KL contribution itself separates as

$$\mathbb{E}_m[D_{\text{KL}}(p \parallel \tilde{p}_m)] = \frac{1}{2} b_m^\top H b_m + \frac{1}{2} \text{tr}(H \Sigma_m) + R_m,$$

where $R_m := \mathbb{E}_m[r]$. Combining this KL decomposition with the linear target-shift term yields the expected excess loss:

$$\mathbb{E}_m[\delta \mathcal{L}] = \underbrace{g^\top b_m}_{D_m: \text{ drift}} + \underbrace{\frac{1}{2} b_m^\top H b_m}_{Q_m: \text{ curvature of drift}} + \underbrace{\frac{1}{2} \text{tr}(H \Sigma_m)}_{V_m: \text{ variance penalty}} + \underbrace{R_m}_{\text{ remainder}}, \quad (9)$$

where D_m is signed; Q_m and V_m are non-negative by $H \succeq 0$. Removing drift is not free: an unbiased mode avoids the drift terms $D_m + Q_m$, but pays the variance penalty V_m .

5.3 Empirical validation of the drift–variance trade-off

We evaluate this trade-off on DistilGPT-2 using $N = 1020$ scored WikiText-2 positions, with reference loss $\mathcal{L}_0 = 4.1354$ nats. The decomposition uses 32 SR seeds at $t = 4, \dots, 10$, with one deterministic RN run per precision. We use bias-corrected quadratic estimators. The logit measurement protocol and the estimators are detailed in [Section B](#).

SR avoids systematic drift but incurs local fluctuation. The decomposition $\Delta_m = b_m + \xi_m$ ([Eq. \(8\)](#)) splits each logit perturbation into drift $b_m = \mathbb{E}_m[\Delta_m]$ and centered fluctuation ξ_m . For RN the split is trivial: $b_{\text{RN}} = \Delta_{\text{RN}}$ and $\xi_{\text{RN}} = 0$, so the perturbation at each token–coordinate pair is entirely deterministic drift. For SR, the perturbation is stochastic, so its expected magnitude is measured by its operational root mean square. All plotted quantities represent the operational statistics of a single forward pass; multiple seeds ($S = 32$) are used exclusively to construct empirical estimators for these quantities.

[Figure 5](#) visualizes the three quantities that enter the loss decomposition ([Eq. \(9\)](#)). Row (i) shows the operational perturbation magnitude per run across all token–coordinate pairs. At the MLP down-projection, RN accumulates deterministic drift, whereas SR’s per-run perturbation is smaller. At the head, however, SR’s per-run perturbation is larger than RN’s. This does not contradict the worst-case bounds of [Theorem 3](#). At the head, inputs are centered by LayerNorm; empirically, we observe that this centering induces cancellation in RN rounding errors, preventing the accumulation of systematic drift. Because the projection is linear, SR guarantees unbiasedness, but incurs higher local variance to do so (local errors up to $1.0u$, compared to RN’s strict $0.5u$ bound). In the absence of systematic drift, the accumulated perturbation is governed by this local variance, which is consistent with SR perturbations being larger than RN at the head.

Nevertheless, not all of this perturbation magnitude translates to excess loss. By shift invariance (Eq. (6)), a per-token uniform shift leaves softmax probabilities unchanged, so only the non-uniform component feeds into Q_m . Row (ii) isolates this loss-relevant component by centering the drift (subtracting each token’s vocabulary mean from b_j). The reduction is pronounced at the MLP: RN’s spread shrinks by an order of magnitude, confirming that most of its accumulation was uniform. SR’s centered drift is near-zero everywhere.

Row (iii) removes the uniform component of the fluctuation (to which the softmax is insensitive by shift invariance). Let $\tilde{\xi}_j = \xi_j - p^\top \xi$ denote the p -centered operational fluctuation; the row reports its standard deviation $\text{std}(\tilde{\xi}_j)$. RN is deterministic ($\xi_{\text{RN}} = 0$). The centered fluctuation is smaller at the MLP than at the head. MLP noise is predominantly a uniform shift of the logits, whereas head noise changes the differences between logits and therefore the softmax output.

The role of input distributions. The accumulation of deterministic rounding errors in inner products depends heavily on the distribution of the operands (El Arar et al., 2023). When inputs are predominantly positive (such as the outputs of the GELU activation at the MLP down-projection), RN rounding errors become biased and, depending on the weight distribution, systematically accumulate into a large drift. Conversely, when inputs are zero-centered (such as the outputs of LayerNorm at the language-model head), positive and negative RN errors tend to cancel, preventing the accumulation of systematic drift.

Loss decomposition at the language-model head. Although SR produces smaller logit drift at the head, it incurs higher excess loss than RN throughout $t = 4, \dots, 10$ (Table 4 and Figure 6). The decomposition in Eq. (9) explains the reversal. RN is deterministic ($V_{\text{RN}} = 0$), so its perturbation contributes only through D_{RN} , Q_{RN} , and R_{RN} . SR’s operational randomness introduces non-uniform fluctuations—the large p -centered fluctuation visible in Figure 5 (iii)—that incur the quadratic cost $V_{\text{SR}} = \frac{1}{2} \text{tr}(H\Sigma_{\text{SR}})$. At the head the noise is genuinely non-uniform across the vocabulary, so the softmax cannot ignore it.

Over $t = 6, \dots, 9$, the variance penalty V_{SR} accounts for 100–105% of SR’s excess loss, while D_{SR} and Q_{SR} lie within 1.6 standard errors of zero. For RN, the perturbation carries no fluctuation ($V_{\text{RN}} = 0$); its non-uniform drift produces $Q_{\text{RN}} = 1.58$ and $R_{\text{RN}} = 0.50$ nats at $t = 6$, for a total excess of 2.12 nats—less than half of SR’s 4.40 nats.

Loss decomposition at the MLP down-projection. At the MLP down-projection, the loss ordering reverses: SR has an excess loss of 0.05 nats against RN’s 0.24 at $t = 6$ (Figure 6 and Table 4). The 3072-length inner product allows deterministic rounding errors under RN to align across many terms, producing a wide, non-uniform drift dispersion that incurs a curvature penalty of $Q_{\text{RN}} = 0.29$ nats.

SR prevents this systematic alignment, keeping Q_{SR} at 0.017 nats. Although SR’s raw fluctuation is comparable to or larger than at the head (6.23 versus 4.27 logits RMS at $t = 6$), the noise has far fewer degrees of freedom. At the head, SR injects rounding errors directly into all 50,257 logits, scrambling the differences between them. At the MLP down-projection, SR injects noise into the 768-dimensional hidden state. Restricted to these 768 degrees of freedom, the 50,257 logits are highly coupled. This dimensionality restriction likely explains why the MLP fluctuation manifests predominantly as a uniform shift across the vocabulary, which the softmax simply ignores. The near-uniform shift also depends on the trained output

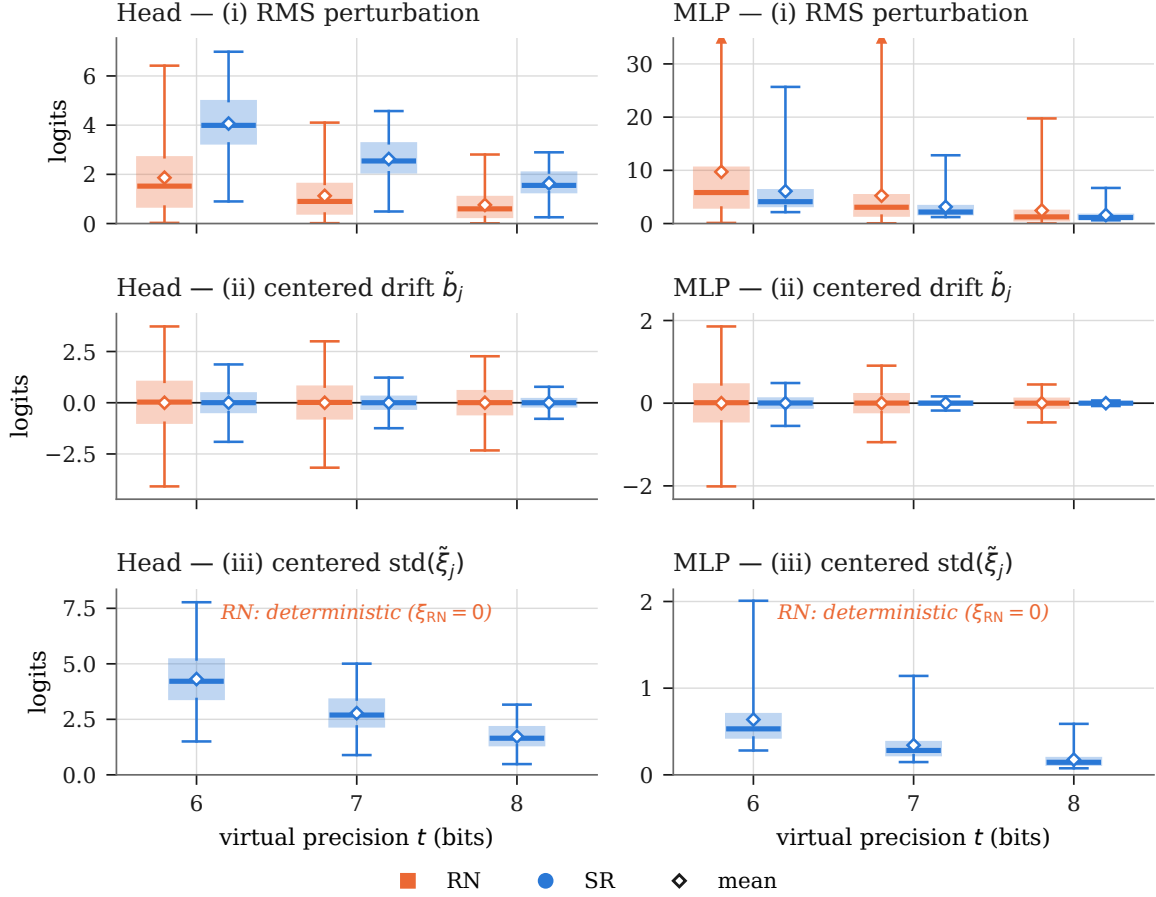


Figure 5: Operational decomposition of logit perturbations across all 51,262,140 token-coordinate pairs (for the MLP, rounding errors are injected at the down-projection and propagated to the final logits). **(i)** Per-run RMS perturbation for pair j : $|\Delta_j|$ for deterministic RN ($\xi_j = 0$), and $\sqrt{\mathbb{E}[\Delta_j^2]} = \sqrt{b_j^2 + \text{std}(\xi_j)^2}$ for SR. **(ii)** Centered drift $\tilde{b}_j$ (per-token vocabulary mean removed): non-uniform component charged via Q_m . **(iii)** Operational standard deviation of the p -centered fluctuation $\text{std}(\tilde{\xi}_j)$, where $\tilde{\xi}_j = \xi_j - p^\top \xi$; RN: $\xi_{RN} = 0$. Boxes IQR, median line, 1st–99th-percentile whiskers; vertical scales differ.

weights and the directions of the propagated error. As a result, the p -centered fluctuation in Figure 5(iii) collapses at the MLP relative to the head, yielding $V_{\text{SR}} = 0.083$ nats, fifty times smaller than at the head. The reduction in drift curvature therefore more than compensates for SR’s variance penalty.

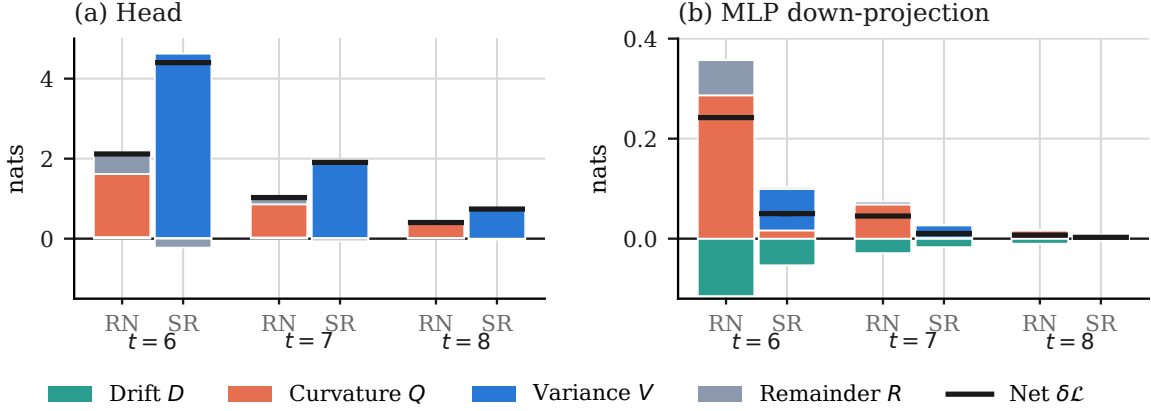


Figure 6: Stacked decomposition of expected excess loss $\mathbb{E}_m[\delta\mathcal{L}] = D_m + Q_m + V_m + R_m$ at the head and MLP down-projection at $t = 6, 7, 8$. Positive components stack upward from zero and negative components downward; solid horizontal marks indicate the measured mean excess loss (with one standard error for SR; RN is deterministic). At the head, the variance penalty V dominates SR excess loss; at the MLP down-projection, RN is dominated by drift curvature Q . Full values are in Table 4.

6 Experiments

Section 5 bounds the forward error of a single reduction and decomposes its effect on the loss at the output softmax. Neither result predicts the perplexity ordering of SR and RN once rounding errors propagate through the network: the bounds are worst-case and per reduction, and the loss decomposition is evaluated one site at a time. This section measures that ordering on DistilGPT-2 at matched significand precision, with the scope of reduced precision ranging from the whole network to one projection in one block (Table 2). The sweeps address three questions: which rounding rule gives lower perplexity at each site, how per-site effects combine across sites and across depth, and whether the per-site preferences yield a mixed-rounding assignment that improves on RN at identical per-site precisions.

6.1 Experimental setup

Model and evaluation benchmark. We evaluate the public `distilbert/distilgpt2` checkpoint (Hugging Face, 2020; Wolf et al., 2020). The architecture retains the layer widths of GPT-2 small ($d_{\text{model}} = 768$, $d_{\text{ff}} = 3072$, $H = 12$ heads, $d_{\text{head}} = 64$, vocabulary $V = 50257$, as illustrated in Figure 1) and distills the depth to six blocks. It therefore has the reduction lengths that enter the bounds of Section 5.1: $n = 768$ for `attn_c_attn`, `attn_c_proj`, and `mlp_c_fc`, and $n = 3072$ for `mlp_c_proj`. Six blocks keep per-operation software emulation tractable. All evaluations use the pretrained weights without fine-tuning, calibration, or quantization-aware adjustment, so that configurations differ only in the arithmetic under test.

Evaluation uses the WikiText-2 test split (Merity et al., 2017). The harness selects the leading 1% of lines, joins them with double newlines, tokenizes the resulting text, and evaluates its first 1024 tokens. These are divided into four non-overlapping contexts of 256 tokens, with context reset between them. Each context contributes 255 next-token targets, giving 1020 scored positions. Perplexity is the exponential of the mean cross-entropy over these positions; the PRISM RN reference at `binary32` significand precision ($t = 24$) is $\text{PPL}_0 = 62.51$. When effects are compared or summed, we also report the excess loss $\ell = \ln(\text{PPL}/\text{PPL}_0)$ in nats, which equals the increase in mean cross-entropy over the 1020 positions and is the quantity decomposed in Eq. (9).

Instrumented arithmetic and scoping. Inference is executed within Fuzzy PyTorch (González-Pepe et al., 2026). During the framework’s compilation, the Verificarlo compiler pass (Denis et al., 2016) intercepts native floating-point operations and redirects them to PRISM’s vectorized C++ backend (Section 4). Instrumented operations within the target scope are rounded to a virtual significand precision $t \leq 24$ using either SR or untied RN, in which half-ULP ties round away from zero rather than to even (Section 4). All SR–RN comparisons in this section use untied RN and not IEEE round-to-nearest with ties-to-even (RNE) at reduced precision. The two differ only on exact ties. RN introduces no randomness, and repeated RN runs are bitwise reproducible.

The harness attaches PyTorch forward pre-hooks that set precision t and the rounding mode immediately before the target module executes, and forward post-hooks that restore $t = 24$ under RN on exit. Inference runs in PyTorch eager mode without cross-module operator fusion. Every operation outside the target scope, including the final loss and perplexity computation, runs at $t = 24$ under RN, so an SR run and its paired RN control differ only in the rounding rule applied within the target scope.

The component and sublayer sweeps reduce precision at the target in all six blocks at once; the depth sweeps reduce it in one block or in a prefix of blocks (Table 2). The `attention` and `mlp` scopes cover every operation inside the corresponding module of each block: the query–key–value and output projections, the score and value products, and the softmax for `attention`; `mlp_c_fc`, the GELU, and `mlp_c_proj` for `mlp`. The `attn_qkav` scope covers only the products $QK^\top$ (reduction length $d_{\text{head}} = 64$) and AV (at most $T = 256$), with the softmax at $t = 24$. The `lm_head` scope is the projection onto the vocabulary logits.

Statistical protocol. The global, component, sublayer, and depth sweeps use five independent seeds per SR configuration, and the mixed-rounding comparison in Section 6.3 uses five seeds per mixed configuration. Results are reported as sample mean $\pm$ standard deviation; for SR, excess loss is averaged over seeds. RN is deterministic at a fixed execution configuration and is run once. We define the perplexity gap as $\Delta = \overline{\text{SR}} - \text{RN}$, so negative values favor SR. We regard the gap as statistically distinguishable from zero when the RN value falls outside the two-sided 95% Student- t confidence interval for the SR mean ($df = 4$). The evaluated text is fixed, so every interval is conditional on the 1020 scored positions and does not describe variation across texts.

6.2 Results

The experiments yield four findings:

Table 2: Sweep levels. All levels compare SR and untied RN at matched significand precision t ; operations outside the target scope run at $t = 24$ under RN.

Level	Target scope	t swept	Question
Global	Whole model	4–14	SR vs. RN comparison under uniform precision
Component	<code>attention</code> , <code>mlp</code> , <code>lm_head</code> , <code>attn_qkav</code> (all blocks)	4–14	Site-dependent sensitivity across main modules
Sublayer	Four projections <code>attn_*</code> , <code>mlp_*</code> (all blocks)	4–14	Impact of reduction length ($n = 768$ vs. 3072)
Depth	<code>mlp_c_proj</code> (single blocks; prefixes $0-k$)	4, 5, 6	Dependence on position; additivity of per-block effects
Mixed recipe	SR at output projections, RN elsewhere	6, 7, 8	Mixed rounding versus all-RN at identical per-site precisions

- (i) **RN is better under uniform rounding.** Reducing only the language-model head reproduces most of the SR–RN gap, suggesting that the head drives the global preference for RN (Figures 7 and 8).
- (ii) **The preferred rounding rule depends on the site.** SR gives lower perplexity in the MLP, whereas RN is preferred at the head and attention input projection. The loss decomposition attributes the MLP advantage to reduced drift and the head disadvantage to SR fluctuations (Figures 8 and 9).
- (iii) **RN loss compounds more strongly across blocks.** For `mlp_c_proj` at $t = 5$ – 6 , RN excess loss exceeds the sum of single-block losses more than SR excess loss does. At $t = 6$, SR is approximately additive (Figure 10).
- (iv) **Mixed-rounding improves on full RN.** Using SR at the attention and MLP output projections and RN elsewhere lowers perplexity, even with one fewer significand bit at the MLP down-projection (Table 3).

Global uniform-precision sweep. Under uniform precision, RN consistently outperforms SR across the entire low-to-moderate precision range, with statistically significant differences at every $t \in [4, 11]$ (Figure 7). At $t = 8$, perplexity is 97.68 under RN and 139.27 ± 3.58 under SR. Reducing only the head reproduces this gap: the SR–RN difference in excess loss is 2.03 nats globally and 2.27 nats with only `lm_head` reduced at $t = 6$, and 0.35 against 0.37 nats at $t = 8$. The RN advantage is therefore consistent with the head’s SR fluctuation penalty (Section 5.3) dominating the whole-network result. This general performance agrees with the forward-pass preference for RN reported by Chmiel et al. (2023), establishing RN as the appropriate default for global precision reduction.

Site performance: MLP versus language-model head. In the MLP, SR gives lower perplexity than RN at every $t \leq 8$, with statistically significant differences (Figure 8). At $t = 6$, perplexity is 71.97 ± 1.16 under SR and 137.93 under RN, a degradation of 15% against 121% over the reference. At $t = 8$, SR is within 0.3% of the reference (62.69) and RN within 2.0% (63.75). At $t \leq 5$ both rules more than double the reference perplexity

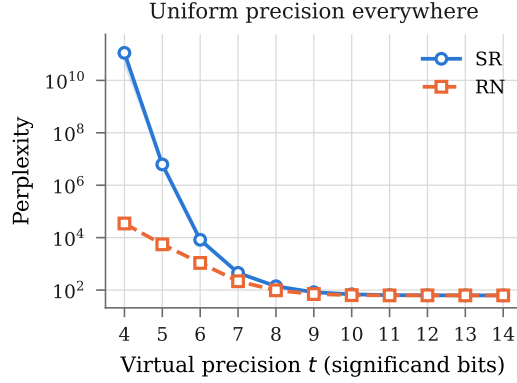


Figure 7: Global sweep at uniform precision across all model operations, showing mean perplexity and SR standard deviation on a logarithmic axis. RN’s advantage is statistically significant across $t \in [4, 11]$.

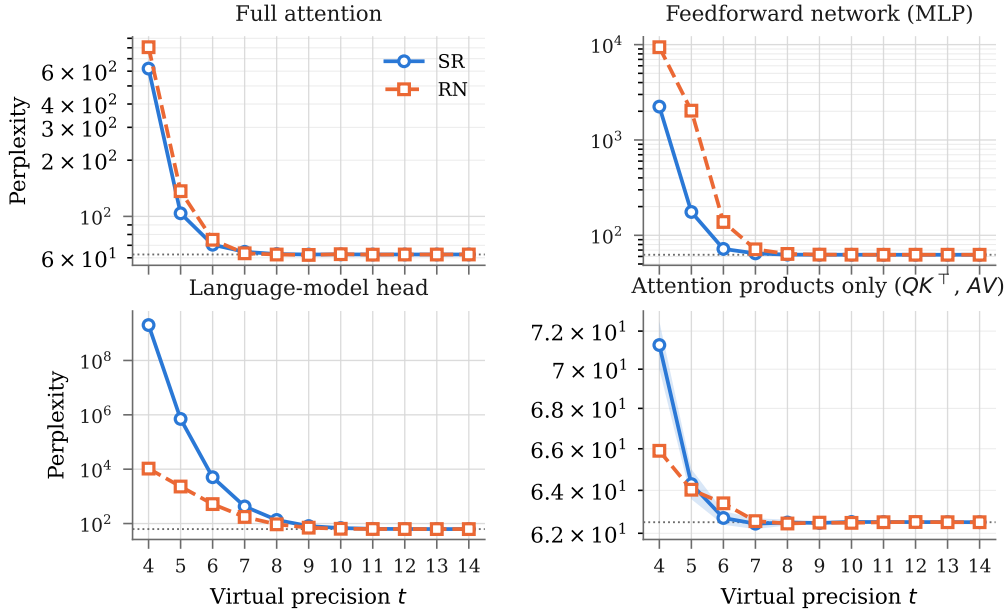


Figure 8: Per-component sweep across precisions $t \in [4, 14]$. Bands show ± 1 sample SD over five SR seeds; dotted line indicates the full-precision reference (62.51).

(at $t = 5$, 175.65 under SR against 2029.83 under RN); the ordering there is statistically significant but concerns configurations that neither rule makes usable.

At the language-model head the ordering reverses: RN gives lower perplexity at every $t \in [4, 12]$, with statistically significant differences. At $t = 8$, perplexity is 135.56 ± 5.22 under SR and 93.60 under RN; at $t = 6$, 5054 ± 464 against 519.18. [Section 5.3](#) decomposes the head’s excess loss at these precisions. Over $t = 6, \dots, 9$ the fluctuation term V_{SR} accounts for

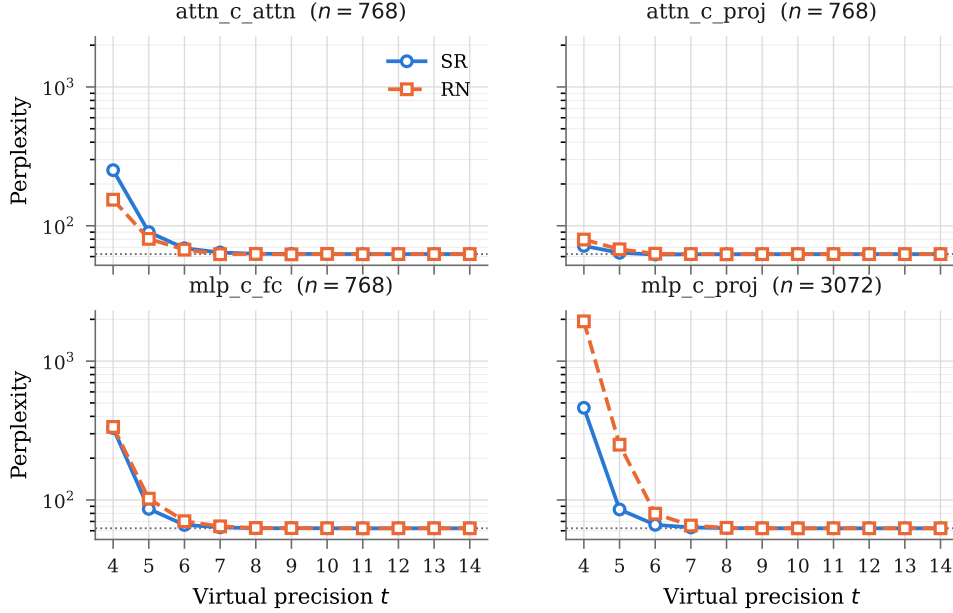


Figure 9: Per-sublayer sweep across the four primary projection matrices, with reduction lengths n indicated and perplexity on a shared logarithmic scale. The feedforward down-projection (`mlp_c_proj`, $n = 3072$) exhibits the largest low-precision SR advantage ($\Delta = -1473.15$ at $t = 4$).

100% of SR’s excess loss, whereas RN, which has no fluctuation, pays only for its non-uniform drift (2.12 against 4.40 nats at $t = 6$). SR noise injected at the head changes the differences between logits, which the cross-entropy penalizes; SR noise injected in the MLP reaches the logits predominantly as a uniform shift, to which the softmax is invariant.

The `attention` module favors SR at $t \leq 6$ but do not favor any rounding mode at higher precisions; at $t \geq 7$ the two rules differ by less than 2% of the reference (at $t = 7$, 64.60 ± 0.49 under SR against 63.42 under RN). Restricted to the score and value products (`attn_qkav`), reduced precision raises perplexity by at most 14% at $t = 4$ (71.26 ± 1.26 under SR, 65.90 under RN) and by at most 2.9% at $t \geq 5$. These products have the shortest reductions in the model ($d_{\text{head}} = 64$ and at most $T = 256$ terms), which may explain their small sensitivity to either rule.

The `attention` module does not decompose into its parts. At $t = 4$, the excess losses of `attn_c_attn`, `attn_c_proj`, and `attn_qkav`, each reduced separately, sum to 1.20 nats under RN and 1.66 nats under SR, which favors RN; reducing the whole module gives 2.56 and 2.29 nats, which favors SR. At $t = 6$ the sums are equal under both rules (0.094 nats), whereas the whole module gives 0.182 under RN and 0.120 under SR.

Reduction length and sublayer sensitivity. Figure 9 separates the four projections, each reduced in all six blocks. The MLP down-projection (`mlp_c_proj`, $n = 3072$) has the largest SR advantage, statistically significant at every $t \leq 7$: at $t = 6$, perplexity is 66.24 ± 0.63 under SR and 79.64 under RN, a degradation of 6.0% against 27%. Among

the three projections with $n = 768$, the preferred rule depends on the site. The attention input projection (`attn_c_attn`) favors RN wherever the difference is statistically significant ($t = 4, 5, 7$; at $t = 4$, 251.58 ± 16.05 under SR against 154.08 under RN). The attention output projection (`attn_c_proj`) favors SR at $t = 4-6$, and the MLP input projection (`mlp_c_fc`) favors SR at $t = 5-6$; at $t \geq 7$ neither difference is statistically significant.

The largest SR advantage occurring at the longest reduction is consistent with [Section 5.1](#): RN rounding errors can align across the n terms of a reduction, whereas SR errors have zero conditional mean, so RN drift can grow faster with n than SR fluctuation. The sweep does not isolate n , however. Only one site has $n = 3072$, and it also differs from the others in its input, the GELU output, which is bounded below by approximately -0.17 . The opposite preferences of three sites with identical $n = 768$ show that factors other than n , such as the input distribution, the weights, and the downstream operations, also determine the ordering.

SR’s advantage across depth. The cumulative sweep in [Figure 10](#) reduces `mlp_c_proj` in progressively longer block prefixes, from block 0 to all six blocks. For every tested precision, SR yields statistically significantly lower perplexity than RN at every prefix, except 0 and 0–1 at $t = 6$, where the differences are not statistically significant.

The advantage of SR becomes more pronounced as more blocks use reduced precision. At $t = 4$, perplexities are initially close (83.24 under SR and 85.13 under RN for block 0), but extending the reduction to all six blocks increases perplexity by a factor of 5.5 under SR and 22.7 under RN. At $t = 6$, the difference becomes statistically significant from prefix 0–2 onward, reaching 66.24 ± 0.63 under SR versus 79.64 under RN with all six blocks reduced, as in [Figure 9](#). These results show that SR limits the perplexity degradation from reducing the MLP down-projection across multiple blocks. The growing SR–RN gap is consistent with RN introducing systematic drift and SR limiting its accumulation across blocks.

This compounding effect illustrates a key structural difference between the rounding rules. The deterministic drift introduced by RN in one block acts as a correlated error that systematically shifts activations in subsequent blocks. In contrast, the noise introduced by SR accumulates without compounding directional drift across depth. While SR’s unbiasedness does not survive non-linear operations, empirically, the downstream propagated noise remains far less biased for SR than for RN, validating its theoretical benefits across network depth.

6.3 Mixed rounding beats full RN at matched precision

The sweeps above reduce one site or one module at a time, and per-site effects do not add ([Section 6.2](#)). We therefore test whether the per-site preferences hold when five sites are reduced together. Each mixed configuration assigns SR or RN per site and is compared with an all-RN configuration at identical per-site precisions ([Table 3](#)).

The assignment follows the sublayer sweeps. SR is used at the two output projections, where it is preferred in isolation: `attn_c_proj` at $t = 6$ (62.10 ± 0.42 under SR against 62.91 under RN) and `mlp_c_proj` at $k \in \{6, 7, 8\}$ (statistically significant at $k = 6, 7$; not statistically significant at $k = 8$). RN is used at the two input projections at $t = 8$, where neither sublayer difference is statistically significant and `attn_c_attn` favors RN at every lower precision with a statistically significant difference. The head runs under RN at $t = 11$, where head-only RN is within 0.8% of the reference (63.00). All remaining operations run at

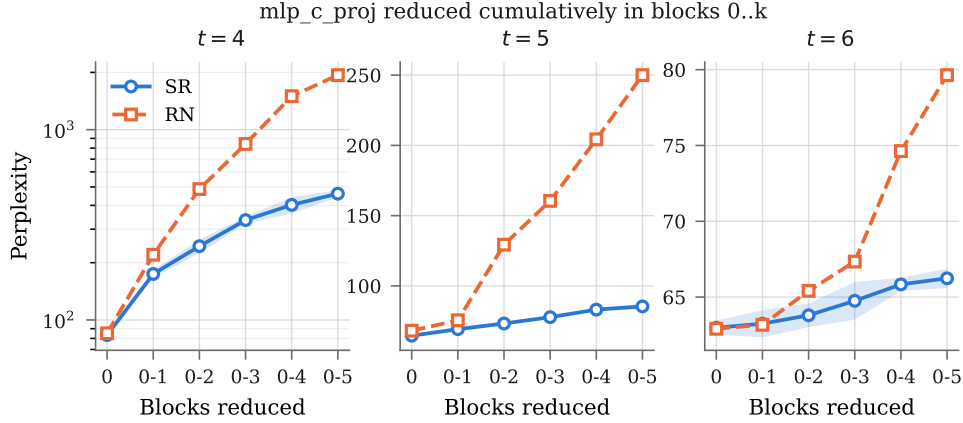


Figure 10: Cumulative propagation across block prefixes 0 to 0- k for `mlp_c_proj` at $t \in \{4, 5, 6\}$. Bands show ± 1 sample SD over five SR seeds.

Table 3: Mixed rounding versus RN at identical per-site significand precisions on WikiText-2. Lower perplexity is better.

(a) Precision and rounding configurations			Configurations apply across all six blocks; $\mu \pm \sigma$ over five seeds. Negative Δ favors mixed rounding. CIs are 95% Student- t ($df = 4$). Relative baseline column shows percentage increase over the full-precision reference (62.51) for Mixed (Full RN).
Site	t bits	Mixed	
Attention input (<code>attn_c_attn</code>)	8	RN	
Attention output (<code>attn_c_proj</code>)	6	SR	
MLP input (<code>mlp_c_fc</code>)	8	RN	
MLP output (<code>mlp_c_proj</code>)	$k \in \{6, 7, 8\}$	SR	
Language-model head (<code>lm_head</code>)	11	RN	
All other operations	24	RN	

(b) Perplexity gaps					
k	Mixed ($\mu \pm \sigma$)	Full RN	Gap Δ (%)	95% CI Δ	Rel. baseline Δ (Full RN)
8	64.63 ± 0.68	66.66	-2.03 (-3.04%)	$[-2.87, -1.18]$	+3.38% (+6.64%)
7	64.89 ± 0.98	68.89	-4.00 (-5.81%)	$[-5.22, -2.79]$	+3.80% (+10.20%)
6	69.01 ± 2.07	96.07	-27.06 (-28.17%)	$[-29.63, -24.49]$	+10.39% (+53.68%)

$t = 24$ under RN. The all-RN control replaces SR by RN at the two output projections and changes nothing else.

These results confirm that combining mixed rounding with mixed precision consistently improves overall performance compared to using RN alone. At identical per-site precisions, the mixed configuration gives lower perplexity than all-RN from 3.04% to 28.17% for all three values of k . The gain grows as the MLP down-projection loses precision, as in the sublayer sweep. Across rows, the mixed configuration at $k = 6$ (69.01 ± 2.07) is not distinguishable from all-RN at $k = 7$ (68.89), and the mixed configuration at $k = 7$ (64.89 ± 0.98) is below all-RN at $k = 8$ (66.66), which lies outside its 95% interval. In this recipe, replacing RN by

SR at the two output projections compensates for removing one significand bit at the MLP down-projection.

6.4 Limitations

PRISM emulates reduced significand precision while retaining the `binary32` exponent range. These results concern significand and accumulator precision under emulation; they do not model the exponent-range restrictions of narrow hardware formats such as FP8 or NVFP4. Transfer to native hardware also depends on its reduction order and arithmetic implementation. Non-target operations in these sweeps use PRISM’s untied RN at $t = 24$, rather than native ties-to-even (RNE); while exact tie events are rare in continuous floating-point activations, their frequency and impact have not been characterized. Incorporating explicit exponent-range limitations remains future work.

The complete sweeps use one six-block model and one leading text prefix. The measured site ordering and composition effects need replication across texts, larger models, and broader tasks. The reduction bounds and loss decomposition identify relevant quantities, but do not establish a universal rounding assignment or quantitative portability of the observed gains.

7 Discussion and conclusion

The rounding rule, when treated as a site-specific design variable rather than a network-wide constant, affects perplexity in opposite directions depending on where it is applied. Inside the feedforward network, SR outperforms RN by a wide margin at low precision, reaching an $11.6\times$ perplexity reduction at $t = 5$. Cumulatively, reducing all six MLP down-projections to $t = 6$ holds perplexity degradation to 6.0% over the full-precision baseline under SR, against 27% under RN (66.24 against 79.64 perplexity). At the language-model head, the ranking reverses and RN yields substantially lower excess loss. While [Chmiel et al. \(2023\)](#) found that RN is generally preferred in the forward pass, and contemporary low-precision formats such as NVIDIA’s NVFP4 ([Abecassis et al., 2025](#)) employ SR selectively for gradient casts during training, our results demonstrate that operation-level granularity is equally consequential at inference: scoping SR specifically to the long reductions of the MLP down-projections mitigates accumulated drift, while retaining RN at the output softmax avoids the severe variance penalty that SR incurs there. These site-specific preferences also enable mixed-rounding recipes: assigning SR to the attention and MLP down-projections and RN to the remaining sites yields relative perplexity reductions of 3.04%–28.17% over matched all-RN configurations across the three tested configurations.

[Section 5](#) identifies which quantities determine the site-level trade-off. An exact decomposition separates the signed shift of the target logit from the relative KL distortion of competing classes; a second-order expansion further splits the distortion into a drift-curvature term Q and a variance penalty V . At the MLP down-projection, RN’s deterministic errors accumulate into a non-uniform logit shift whose curvature cost exceeds SR’s variance penalty, which is small because the propagated SR noise is predominantly a uniform vocabulary shift that the softmax ignores. At the language-model head, SR’s fluctuation ξ is genuinely non-uniform and is penalized directly by cross-entropy. The decomposition establishes that the comparative advantage of either rounding rule is governed not by raw error magnitude,

but by whether the resulting perturbations propagate as a uniform vocabulary shift or disperse as non-uniform distortion across logits.

Establishing the broader generality of this trade-off requires evaluating the decomposition across larger models, varied architectures, and diverse downstream tasks. Porting PRISM directly to GPUs (where an early prototype is in development) and incorporating exponent-range checks into variable-precision frameworks will be essential steps to scale this analysis to modern billion-parameter architectures and complete narrow hardware formats.

Reproducibility. All evaluation pipelines, experiment scripts, and aggregated data are publicly available at <https://github.com/big-data-lab-team/fuzzy-llm> and archived on Zenodo ([doi:10.5281/zenodo.23066028](https://doi.org/10.5281/zenodo.23066028)).

Acknowledgments and Disclosure of Funding

This work was partially funded by the FPT-4 (ANR-24-CE46-7572) project.

Appendix A. Proof of Exactness of the Rounding Decision

This appendix provides the complete proof of [Theorem 1](#) from [Section 3.1](#), establishing that the floating-point evaluation of the decision boundary in [Algorithm 1](#) preserves the exact mathematical sign under Hypotheses 1–3.

Sign normalization. Define the sign-normalized variables $\rho' = \rho \cdot s_{\text{dir}}$, $\pi' = \pi \cdot s_{\text{dir}}$, and $\tau' = \tau \cdot s_{\text{dir}}$. Since $s_{\text{dir}} \in \{-1, 1\}$, the theorem’s conclusion is equivalent to

$$(\rho' - \pi') + \tau' \geq 0 \iff \text{fl}(\text{fl}(\rho' - \pi') + \tau') \geq 0.$$

When $\rho \neq 0$, $s_{\text{dir}} = \text{sgn}(\rho)$, so $\rho' = |\rho| \geq 0$ and $\pi' = |\pi| \geq 0$ (since ρ and π share the same sign by Hypothesis 3). When $\rho = 0$, $s_{\text{dir}} = \text{sgn}(\tau)$, so $\rho' = 0$ and $\tau' = |\tau| \geq 0$.

Lemma 4 (No collapse to zero) *Under Hypotheses 1–3 of [Theorem 1](#), if $\rho' \geq \text{ulp}_{\text{hw}}$ and $(\rho' - \pi') + \tau' < 0$, then $\text{fl}(\text{fl}(\rho' - \pi') + \tau') \neq 0$.*

Proof Assume for contradiction that the evaluated result collapses to zero: $\text{fl}(\text{fl}(\rho' - \pi') + \tau') = 0$. An IEEE 754 addition rounds to 0 if and only if the exact sum is 0. Thus, $\text{fl}(\rho' - \pi') + \tau' = 0 \implies \text{fl}(\rho' - \pi') = -\tau'$. We now establish the conditions for Sterbenz’s Lemma ([Sterbenz, 1974](#)) ($\frac{\rho'}{2} \leq \pi' \leq 2\rho'$):

- **Lower bound** ($\pi' \geq \frac{\rho'}{2}$): Our starting assumption $(\rho' - \pi') + \tau' < 0$ rearranges directly to $\pi' > \rho' + \tau'$. By Hypotheses 1 and 2, $\tau' \geq -\text{ulp}_{\text{hw}}/2 \geq -\rho'/2$. Therefore, $\pi' > \rho' - \rho'/2 = \frac{\rho'}{2}$.
- **Upper bound** ($\pi' \leq 2\rho'$): The absolute rounding error of the subtraction is bounded by $\frac{1}{2}\text{ulp}_{\text{hw}}(-\tau')$. By Hypothesis 1, $|\tau'| \leq \text{ulp}_{\text{hw}}/2$, and $\text{ulp}_{\text{hw}}(\cdot)$ is nondecreasing in magnitude, so $\frac{1}{2}\text{ulp}_{\text{hw}}(-\tau') \leq \frac{1}{2}\text{ulp}_{\text{hw}}(\text{ulp}_{\text{hw}}/2) \leq \text{ulp}_{\text{hw}}/2$. Thus, $|\text{fl}(\rho' - \pi') - (\rho' - \pi')| = |-\tau' - (\rho' - \pi')| = |(\rho' - \pi') + \tau'| \leq \text{ulp}_{\text{hw}}/2$. Applying the reverse triangle inequality yields $|\rho' - \pi'| - |\tau'| \leq \text{ulp}_{\text{hw}}/2$. Since $|\tau'| \leq \text{ulp}_{\text{hw}}/2$ and $\text{ulp}_{\text{hw}} \leq \rho'$, we obtain $|\rho' - \pi'| \leq \text{ulp}_{\text{hw}} \leq \rho'$. This bound implies $\rho' - \pi' \geq -\rho'$, which gives $\pi' \leq 2\rho'$.

Sterbenz’s Lemma guarantees the subtraction is exact: $\text{fl}(\rho' - \pi') = \rho' - \pi'$. Substituting this exactness back into our derivation yields $(\rho' - \pi') + \tau' = 0$, which contradicts our starting assumption that $(\rho' - \pi') + \tau' < 0$. Therefore, the collapse to 0 is impossible. ■

Proof of Theorem 1.

Case $\rho = 0$. When $\rho' = 0$, the subtraction $\text{fl}(\rho' - \pi') = \text{fl}(-\pi') = -\pi'$ is exact, since negation is exact in IEEE 754. The evaluation reduces to a single rounding $\text{fl}(-\pi' + \tau')$. For the forward direction ($\implies$), assume $(\rho' - \pi') + \tau' \geq 0$; then $\text{fl}(-\pi' + \tau') \geq 0$ by monotonicity. For the backward direction ($\impliedby$), assume $\text{fl}(-\pi' + \tau') \geq 0$ but, for contradiction, $(\rho' - \pi') + \tau' < 0$. Monotonicity forces $\text{fl}(-\pi' + \tau') \leq 0$, hence the result is exactly 0. But an IEEE 754 addition of representable inputs rounds to 0 only when the exact sum is 0, a contradiction.

Case $\rho \neq 0$. Hypothesis 2 gives $\rho' = |\rho| \geq \text{ulp}_{\text{hw}}$. For the forward direction ($\implies$), assume $(\rho' - \pi') + \tau' \geq 0$, i.e., $\rho' - \pi' \geq -\tau'$. By monotonicity, $\text{fl}(\rho' - \pi') \geq \text{fl}(-\tau') = -\tau'$, so $\text{fl}(\rho' - \pi') + \tau' \geq 0$. Applying monotonicity once more, $\text{fl}(\text{fl}(\rho' - \pi') + \tau') \geq 0$. For the backward direction ($\impliedby$), assume $\text{fl}(\text{fl}(\rho' - \pi') + \tau') \geq 0$ but, for contradiction, $(\rho' - \pi') + \tau' < 0$. By the same monotonicity argument with reversed inequalities, $\text{fl}(\text{fl}(\rho' - \pi') + \tau') \leq 0$. Combined with the assumption, the floating-point result is exactly 0. But Lemma 4 shows this collapse is impossible, a contradiction. ■

Appendix B. Drift and variance measurements

Table 4 provides the numerical decomposition behind Section 5.3. All estimates condition on the same $N = 1020$ scored positions.

Logit measurement protocol. The evaluation uses the same DistilGPT-2 checkpoint and $N = 1020$ scored WikiText-2 positions as the perplexity sweeps (Section 6.1), with full-precision reference loss $\mathcal{L}_0 = 4.1354$ nats ($\text{PPL}_0 = 62.51$). Unlike the RN-background sweeps in Section 6.1, instrumented operations outside the lowered site use the matching rounding rule at $t = 24$ (SR background for SR; RN for RN). The SR perturbations therefore include full-precision background rounding; an eight-seed SR control with no lowered site yields mean excess loss of $(6.66 \pm 14.48) \times 10^{-8}$ nats, verifying that the background contribution is negligible. As noted in Section 5.3, the decomposition estimates condition on this text using $S = 32$ SR seeds and one deterministic RN run per precision ($t = 4, \dots, 10$).

Finite-seed estimators. With $\hat{b} = S^{-1} \sum_s \Delta_s$, $A = \frac{1}{2S} \sum_s \text{Var}_p(\Delta_s)$, and $B = \frac{1}{2} \text{Var}_p(\hat{b})$, the unbiased quadratic estimators are $\hat{Q} = \frac{SB-A}{S-1} = B - \frac{\hat{V}}{S}$ and $\hat{V} = \frac{S(A-B)}{S-1}$. We use $\hat{D} = g^\top \hat{b}$ and define $\hat{R}$ by subtracting these three terms from the measured mean excess loss; agreement of their sum is therefore not an independent validation. Standard errors are estimated by leaving out each seed in turn and recomputing the estimator. Although $Q \geq 0$, its unbiased estimate can be negative: at the head and $t = 8$, $\hat{Q}$ is -0.0017 ± 0.0013 at $S = 8$ and -0.0003 ± 0.0004 at $S = 32$. Both estimates lie within two standard errors of zero. We compute the variance penalty without forming the full covariance matrix via $2V = \text{tr}(H\Sigma) = \sum_j p_j \Sigma_{jj} - \text{Var}_s(p^\top \Delta)$, which requires only the coordinate variances and the variance of $p^\top \Delta$.

Table 4: Loss decomposition terms (Eq. (9)) in nats, averaged over $N = 1020$ positions. SR reports mean ± 1 SE ($S = 32$ seeds); RN is deterministic ($V_{\text{RN}} = 0$).

	t	D		Q		V		R		$\mathbb{E}[\delta\mathcal{L}]$	
lm_head ($n = 768$)	SR	4	0.049 $\pm$ 0.043	-0.002 $\pm$ 0.011	23.204 $\pm$ 0.047	-5.860 $\pm$ 0.046	17.392 $\pm$ 0.045				
		5	-0.009 $\pm$ 0.021	-0.001 $\pm$ 0.006	10.446 $\pm$ 0.021	-1.141 $\pm$ 0.022	9.295 $\pm$ 0.025				
		6	0.015 $\pm$ 0.016	0.002 $\pm$ 0.002	4.606 $\pm$ 0.009	-0.223 $\pm$ 0.009	4.400 $\pm$ 0.016				
		7	0.016 $\pm$ 0.010	0.001 $\pm$ 0.001	1.951 $\pm$ 0.003	-0.062 $\pm$ 0.003	1.907 $\pm$ 0.011				
		8	-0.008 $\pm$ 0.008	-0.0003 $\pm$ 0.0004	0.766 $\pm$ 0.001	-0.017 $\pm$ 0.001	0.740 $\pm$ 0.007				
		9	0.003 $\pm$ 0.003	0.000 $\pm$ 0.000	0.270 $\pm$ 0.001	-0.003 $\pm$ 0.000	0.271 $\pm$ 0.003				
		10	0.003 $\pm$ 0.002	0.000 $\pm$ 0.000	0.084 $\pm$ 0.000	0.000 $\pm$ 0.000	0.087 $\pm$ 0.002				
	RN	4	-0.016	3.166	0	1.974	5.124				
		5	0.104	2.402	0	1.100	3.605				
		6	0.041	1.576	0	0.500	2.117				
		7	0.030	0.830	0	0.168	1.028				
		8	0.019	0.346	0	0.038	0.404				
		9	-0.012	0.124	0	0.007	0.120				
		10	-0.014	0.039	0	0.000	0.025				
mlp_c_proj ($n = 3072$)	SR	4	-0.2553 $\pm$ 0.0064	0.9690 $\pm$ 0.0042	0.4566 $\pm$ 0.0020	0.8280 $\pm$ 0.0062	1.9983 $\pm$ 0.0085				
		5	-0.1300 $\pm$ 0.0047	0.1645 $\pm$ 0.0022	0.2245 $\pm$ 0.0012	0.0680 $\pm$ 0.0024	0.3270 $\pm$ 0.0054				
		6	-0.0532 $\pm$ 0.0026	0.0165 $\pm$ 0.0005	0.0828 $\pm$ 0.0005	0.0040 $\pm$ 0.0004	0.0501 $\pm$ 0.0027				
		7	-0.0169 $\pm$ 0.0019	0.0014 $\pm$ 0.0001	0.0252 $\pm$ 0.0002	0.0006 $\pm$ 0.0001	0.0104 $\pm$ 0.0018				
		8	-0.0043 $\pm$ 0.0008	0.0001 $\pm$ 0.0000	0.0067 $\pm$ 0.0001	0.0001 $\pm$ 0.0000	0.0026 $\pm$ 0.0007				
		9	-0.0013 $\pm$ 0.0004	0.0000 $\pm$ 0.0000	0.0017 $\pm$ 0.0000	0.0000 $\pm$ 0.0000	0.0005 $\pm$ 0.0004				
		10	-0.0004 $\pm$ 0.0002	0.0000 $\pm$ 0.0000	0.0004 $\pm$ 0.0000	0.0000 $\pm$ 0.0000	0.0001 $\pm$ 0.0002				
	RN	4	-0.1676	2.4204	0	1.1792	3.4321				
		5	-0.1250	1.0504	0	0.4607	1.3861				
		6	-0.1148	0.2867	0	0.0702	0.2421				
		7	-0.0288	0.0686	0	0.0055	0.0453				
		8	-0.0101	0.0166	0	0.0006	0.0071				
		9	-0.0043	0.0040	0	0.0000	-0.0002				
		10	-0.0020	0.0010	0	0.0000	-0.0009				

References

- F. Abecassis, A. Agrusa, D. Ahn, J. Alben, et al. Pretraining large language models with NVFP4. *arXiv preprint arXiv:2509.25149*, 2025.
- Amazon Web Services. NeuronCore-v2 architecture. Neuron Documentation, <https://awsdocs-neuron.readthedocs-hosted.com>, 2023.
- S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefer, and J. Hensman. QuaRot: Outlier-free 4-bit inference in rotated LLMs. In *Advances in Neural Information Processing Systems*, volume 37, pages 100213–100240, 2024. doi: 10.52202/079017-3180.
- S. Ben Ali, S.-I. Filip, and O. Sentieys. A stochastic rounding-enabled low-precision floating-point MAC for DNN training. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1–6. IEEE, 2024. doi: 10.23919/DATE58400.2024.10546735.
- T. Beuzeville, A. Buttari, S. Gratton, and T. Mary. Deterministic and probabilistic rounding error analysis of neural networks in floating-point arithmetic. *IMA Journal of Numerical Analysis*, 2026. doi: 10.1093/imanum/draf130.
- P. Blanchard, D. J. Higham, and N. J. Higham. Accurately computing the log-sum-exp and softmax functions. *IMA Journal of Numerical Analysis*, 41(4):2311–2330, 2021. doi: 10.1093/imanum/draa038.
- S. Boldo and J.-M. Muller. Exact and approximated error of the FMA. *IEEE Transactions on Computers*, 60(2):157–164, 2011. doi: 10.1109/TC.2010.139.
- Y. Chatelain. PRISM: Probabilistic rounding with instruction set management. <https://github.com/verificarlo/prism>, 2026.
- B. Chmiel, R. Banner, E. Hoffer, H. Ben-Yaacov, and D. Soudry. Accurate neural training with 4-bit matrix multiplications at standard formats. In *International Conference on Learning Representations*, 2023.
- M. P. Connolly, N. J. Higham, and T. Mary. Stochastic rounding and its probabilistic backward error analysis. *SIAM Journal on Scientific Computing*, 43(1):A566–A585, 2021. doi: 10.1137/20M1334796.
- M. Croci, M. Fasi, N. J. Higham, T. Mary, and M. Mikaitis. Stochastic rounding: implementation, error analysis and applications. *Royal Society Open Science*, 9(3):211631, 2022. doi: 10.1098/rsos.211631.
- P. de Oliveira Castro, E.-M. El Arar, E. Petit, and D. Sohier. Error analysis of sum-product algorithms under stochastic rounding. *SIAM Journal on Scientific Computing*, 47(6): B1481–B1502, 2025. doi: 10.1137/24M1710966.
- T. J. Dekker. A floating-point technique for extending the available precision. *Numerische Mathematik*, 18(3):224–242, 1971. doi: 10.1007/BF01397083.

- C. Denis, P. de Oliveira Castro, and E. Petit. Verificarlo: checking floating point accuracy through Monte Carlo arithmetic. In *2016 IEEE 23rd Symposium on Computer Arithmetic (ARITH)*, pages 55–62, 2016. doi: 10.1109/ARITH.2016.31.
- T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22*, pages 30318–30332, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088. doi: 10.52202/068431-2198.
- E.-M. El Arar, D. Sohier, P. de Oliveira Castro, and E. Petit. Stochastic rounding variance and probabilistic bounds: A new approach. *SIAM Journal on Scientific Computing*, 45(5):C255–C275, 2023. doi: 10.1137/22M1510819.
- E.-M. El Arar, D. Sohier, P. de Oliveira Castro, and E. Petit. Bounds on non-linear errors for variance computation with stochastic rounding. *SIAM Journal on Scientific Computing*, 46(5):B579–B599, 2024. doi: 10.1137/23M1563001.
- M. Fasi and M. Mikaitis. Algorithms for stochastically rounded elementary arithmetic operations in IEEE 754 floating-point arithmetic. *IEEE Transactions on Emerging Topics in Computing*, 9(3):1451–1466, 2021. doi: 10.1109/TETC.2021.3069165.
- F. F  votte and B. Lathuili  re. VERROU: Assessing floating-point accuracy without recompiling. Technical report, EDF R&D, 2016. HAL: hal-01383417.
- G. E. Forsythe. Reprint of a note on rounding-off errors. *SIAM Review*, 1(1):66–67, 1959. doi: 10.1137/1001011.
- Z. Gong, J. Liu, J. Wang, X. Cai, D. Zhao, and R. Yan. What makes quantization for large language model hard? an empirical study from the lens of perturbation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):18082–18089, 2024. doi: 10.1609/aaai.v38i16.29765.
- I. Gonz  lez-Pepe, H. Akhaddar, T. Glatard, and Y. Chatelain. Fuzzy PyTorch: Rapid numerical variability evaluation for deep learning models. *Transactions on Machine Learning Research*, 2026. arXiv:2605.25991.
- Google. Highway: Efficient and portable SIMD/vector C++ library. <https://github.com/google/highway>, 2024.
- S. Graillat and J.-M. Muller. Emulation of the FMA in rounded-to-nearest floating-point arithmetic. *Journal of Computational and Applied Mathematics*, 433:115304, 2023. doi: 10.1016/j.cam.2023.115304.
- S. Graillat, P. Langlois, and N. Louvet. Improving the compensated Horner scheme with a fused multiply and add. In *Proceedings of the 2006 ACM Symposium on Applied Computing*, pages 1323–1327, 2006. doi: 10.1145/1141277.1141585.
- Graphcore. AI-Float: Mixed-precision arithmetic for AI, a hardware perspective. Technical white paper, <https://docs.graphcore.ai>, 2020.

- S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan. Deep learning with limited numerical precision. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1737–1746. PMLR, 2015.
- D. Hendrycks and K. Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.
- N. J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, Philadelphia, PA, 2nd edition, 2002. doi: 10.1137/1.9780898718027.
- N. J. Higham and T. Mary. A new approach to probabilistic rounding error analysis. *SIAM Journal on Scientific Computing*, 41(5):A2815–A2835, 2019. doi: 10.1137/18M1226312.
- Hugging Face. DistilGPT2 model card. <https://huggingface.co/distilbert/distilgpt2>, 2020.
- IEEE. IEEE standard for floating-point arithmetic. IEEE Std 754-2019, 2019.
- I. C. F. Ipsen and H. Zhou. Probabilistic error analysis for inner products. *SIAM Journal on Matrix Analysis and Applications*, 41(4):1726–1741, 2020. doi: 10.1137/19M1270434.
- S. Kim, C. Hooper, A. Gholami, Z. Dong, X. Li, S. Shen, M. W. Mahoney, and K. Keutzer. SqueezeLLM: Dense-and-sparse quantization. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 23901–23923. PMLR, 2024.
- J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems*, volume 6, pages 87–100, 2024.
- J. Martens. New insights and perspectives on the natural gradient method. *Journal of Machine Learning Research*, 21(146):1–76, 2020. URL <https://www.jmlr.org/papers/v21/17-678.html>.
- S. Merity, C. Xiong, J. Bradbury, and R. Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017.
- J.-M. Muller, N. Brunie, F. de Dinechin, C.-P. Jeannerod, M. Joldes, V. Lefèvre, G. Melquiond, N. Revol, and S. Torres. *Handbook of Floating-Point Arithmetic*. Birkhäuser, 2nd edition, 2018. doi: 10.1007/978-3-319-76526-6.
- M. Nagel, R. A. Amjad, M. van Baalen, C. Louizos, and T. Blankevoort. Up or down? Adaptive rounding for post-training quantization. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 7197–7206. PMLR, 2020.
- NVIDIA. Parallel thread execution ISA: The `cvt.rs` stochastic-rounding instruction. CUDA Documentation, <https://docs.nvidia.com/cuda>, 2025.

- D. S. Parker. Monte carlo arithmetic: exploiting randomness in floating-point arithmetic. Technical Report CSD-970002, University of California, Los Angeles, 1997.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.
- P. H. Sterbenz. *Floating-Point Computation*. Prentice-Hall, Englewood Cliffs, NJ, 1974.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.
- T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020. doi: 10.18653/v1/2020.emnlp-demos.6.
- G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 38087–38099. PMLR, 2023.
- R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T.-Y. Liu. On layer normalization in the transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 10524–10533. PMLR, 2020.
- T. Zhang, Z. Lin, G. Yang, and C. De Sa. QPyTorch: A low-precision arithmetic simulation framework. In *2019 Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing - NeurIPS Edition (EMC2-NIPS)*, pages 10–13, 2019. doi: 10.1109/EMC2-NIPS53020.2019.00010.